SMARTCRICKET AI: AN EXPLAINABLE MULTIMODAL CRICKET SHOT ANALYSIS AND INTELLIGENT VIRTUAL COACHING SYSTEM

A PROJECT REPORT

Submitted by

PRABHU K 510925405009

In partial fulfillment for the award of the degree

Of

MASTER OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

GLOBAL INSTITUTE OF ENGINEERING AND TECHNOLOGY

MELVISHARAM

ANNA UNIVERSITY: CHENNAI 600 025 NOV 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

This is to certify that the project title "**SMARTCRICKET AI: AN EXPLAINABLE MULTIMODAL CRICKET SHOT ANALYSIS AND INTELLIGENT VIRTUAL COACHING SYSTEM**" is the Bonafide work of **(PRABHU K - 510925405009)** who carried out the project work under my supervision. Certified further, the work reported herein does not form a part of any other project work on the basis of which a degree or award was conferred earlier on this title.

SIGNATURE	**SIGNATURE**
[SUPERVISOR NAME]	Dr. D. UDAYASURIYA RAJKUMAR, M.E(CSE)., PhD.,
SUPERVISOR	
[Designation]	**HEAD OF THE DEPARTMENT**
Computer Science and Engineering	Professor
Global Institute of Engineering and Technology, Ranipet-632509	Computer Science and Engineering
	Global Institute of Engineering and Technology, Ranipet-632509

Submitted for project viva voce examination held on ……

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I am very much indebted to our beloved Chairperson **Mrs. HEMAPRASAD**, Chairman **Mr. RAMAPRASAD VEDULA** and Managing Director **Dr. SUHAS VEDULA** for providing the facilities to carry out this project.

I express my sincere thanks to our Principal **Dr. V. NATARAJAN, M.E., PhD., FIE**, for the opportunity given to me.

I would like to express my deep sense of gratitude to the Head of the Department **Dr. D. UDAYASURIYA RAJKUMAR, M.E., PhD.**, who has always been a mentor and motivator to complete this project.

I would like to extend my deep sense of gratitude to my Guide **[SUPERVISOR NAME]**, for the valuable guidance given throughout the project.

I also thank the **Teaching and Non-Teaching staff members** of the Department of **Computer Science and Engineering** for their constant support.

I acknowledge M. K. Dihan, Abdullah, Amina and S. Ahmed, the authors of CricShot10k, for releasing the dataset and code, which made this work possible.

With a deep sense of reverence, I express my gratitude to my parents and family members, who have always supported me morally as well as economically.

Signature of the Candidate
(PRABHU K)

ABSTRACT

Cricket coaching and broadcast analysis still depend on people watching every ball and tagging each batting shot by hand. The work is slow, it does not scale across leagues and formats, and two annotators often label the same shot differently. Automatic recognition of batting shots from video is therefore a useful but difficult computer vision problem: shots are short, visually similar and filmed under changing camera angles, lighting and jersey colours.

This project, SmartCricket AI, designs and implements an explainable, multimodal system for cricket shot analysis with an intelligent virtual coach. It builds on the CricShot10k dataset of 10,086 one-second broadcast clips in 15 shot classes and its CricShotNet model (EfficientNetV2-S with a 128-unit GRU), which reports 89% top-1 accuracy. In image mode, a photograph of a batter is passed to MediaPipe Pose, and the 33 detected landmarks are turned into a posture report of joint angles and visibility scores. In video mode, a clip is sampled to 15 frames and classified by video-only, pose-only and fused video and pose models trained on the same split.

Each prediction carries a temperature-scaled confidence, a top-3 list, Grad-CAM heatmaps, an attention-based key-frame timeline and a Dynamic Time Warping comparison with a reference clip. A large language model, given only the stored results, acts as a virtual coach, and an adaptive training coach plans daily drills. The platform uses HTML5, CSS3, JavaScript, FastAPI, PyTorch and PostgreSQL.

On 1,289 test clips, fusing video with 2D pose raised top-1 accuracy from 56.2% to 63.9% and macro-F1 from 55.3% to 62.2%, a statistically significant gain (McNemar test, $p < 10^{-8}$). Temperature scaling lowered the calibration error of all models.

Keywords: cricket shot classification, MediaPipe Pose, Grad-CAM, temperature scaling, Dynamic Time Warping, large language model

TABLE OF CONTENTS

Chapter No	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF FIGURES	v
	LIST OF TABLES	vi
	LIST OF ABBREVIATIONS	vii
1	**INTRODUCTION**	**1**
	1.1 GENERAL	1
	1.2 BATTING STROKES IN BRIEF	2
	1.3 MOTIVATION	3
	1.4 PROBLEM STATEMENT	4
	1.5 OBJECTIVES	4
	1.6 SCOPE OF THE PROJECT	5
	1.7 ORGANISATION OF THE REPORT	6
2	**LITERATURE SURVEY**	**7**
	2.1 EXISTING SYSTEM	7
	2.1.1 Recognising shots from single images	7
	2.1.2 Recognising shots from video	8
	2.1.3 CricShot10k and the CricShotNet pipeline	8
	2.1.4 Pose estimation	10
	2.1.5 Comparison of related work	11
	2.2 DISADVANTAGES	11
3	**PROPOSED SYSTEM**	**14**
	3.1 ARCHITECTURE OF THE PROPOSED METHOD	14
	3.2 IMAGE MODE	15
	3.3 VIDEO MODE	16
	3.4 AI VIRTUAL COACH AND PLATFORM	16
	3.5 KEY INNOVATIONS	17
4	**SYSTEM REQUIREMENTS**	**18**
	4.1 HARDWARE REQUIREMENTS	18
	4.2 SOFTWARE REQUIREMENTS	18
	4.3 SOFTWARE DESCRIPTION	19
	4.3.1 Python	19
	4.3.2 PyTorch and torchvision	19

Chapter No	TITLE	PAGE NO
	4.3.3 MediaPipe and OpenCV	19
	4.3.4 FastAPI and Uvicorn	20
	4.3.5 PostgreSQL and SQLAlchemy	20
	4.3.6 HTML5, CSS3, JavaScript and Chart.js	20
	4.3.7 Google Colab	20
	4.3.8 Large language model API	20
5	**SYSTEM DESIGN**	**21**
	5.1 DATA FLOW DIAGRAM	21
	5.1.1 Level 0 data flow diagram	21
	5.1.2 Level 1 data flow diagram	21
	5.2 USE CASE DIAGRAM	22
	5.3 CLASS DIAGRAM	23
	5.4 SEQUENCE DIAGRAM	23
	5.5 ENTITY RELATIONSHIP DIAGRAM	24
	5.6 COLLABORATION DIAGRAM	26
6	**MODULE DESCRIPTION**	**27**
	6.1 USER MANAGEMENT, HISTORY AND ADMINISTRATION	27
	6.2 PHOTO POSTURE ANALYSIS	27
	6.3 VIDEO PREPROCESSING AND SHOT CLASSIFICATION	29
	6.4 MOVEMENT COMPARISON AND EXPLAINABLE AI	31
	6.5 AI VIRTUAL COACH AND ADAPTIVE TRAINING	32
7	**TESTING**	**35**
	7.1 TESTING STRATEGY	35
	7.2 UNIT TESTING	36
	7.3 INTEGRATION TESTING	37
	7.4 SYSTEM TESTING	38
	7.5 ACCEPTANCE TESTING	38
	7.6 PERFORMANCE EVALUATION OF THE SHOT CLASSIFIERS	39
	7.6.1 Classification results	39
	7.6.2 Per-class analysis	41
	7.6.3 Calibration	42
	7.6.4 Robustness across camera views and sharpness	43

Chapter No	TITLE	PAGE NO
	7.7 TEST SUMMARY	44
8	**ADVANTAGES OF THE PROPOSED SYSTEM**	**45**
9	**CONCLUSION AND FUTURE WORK**	**47**
	9.1 CONCLUSION	47
	9.2 LIMITATIONS	48
	9.3 FUTURE WORK	48
	9.4 WORK SCHEDULE FOR PHASE II	49
	APPENDIX 1: SAMPLE CODINGS	**50**
	APPENDIX 2: SNAP SHOTS	**52**
	REFERENCES	**56**

LIST OF FIGURES

FIGURE NUMBER	FIGURE	PAGE NUMBER
1.1	A BATTER MOVING ONTO THE FRONT FOOT TO MEET THE BALL	3
2.1	PREPROCESSING IN CRICSHOTNET: CROPPED FRAME (LEFT) AND THE SAME FRAME WITH THE BAT AND BATTER SEGMENTED (RIGHT)	9
2.2	CRICSHOTNET PIPELINE OF THE EXISTING SYSTEM (FROM [1])	10
3.1	OVERVIEW OF THE PROPOSED SMARTCRICKET AI SYSTEM	14
3.2	LAYERED ARCHITECTURE OF SMARTCRICKET AI	15
3.3	CLASSIFIER VARIANTS COMPARED IN EXPERIMENTS E1 TO E3; E9 REPEATS THE POSE AND FUSION MODELS WITH 3D POSE	16
5.1	LEVEL 0 DATA FLOW DIAGRAM (CONTEXT DIAGRAM)	21
5.2	LEVEL 1 DATA FLOW DIAGRAM	22
5.3	USE CASE DIAGRAM	22
5.4	CLASS DIAGRAM	23
5.5	SEQUENCE DIAGRAM FOR VIDEO ANALYSIS	24
5.6	ENTITY RELATIONSHIP DIAGRAM	25
5.7	COLLABORATION DIAGRAM FOR A VIDEO ANALYSIS AND A COACH QUESTION	26
6.1	LANDMARKS AND JOINT ANGLES USED BY THE POSTURE REPORT AND COMPARISON	28
7.1	LEVELS OF TESTING USED IN THE PROJECT	35
7.2	CONFUSION MATRIX OF THE BEST MODEL ON THE TEST SPLIT; ROWS ARE TRUE SHOTS, COLUMNS PREDICTED SHOTS	42
7.3	RELIABILITY DIAGRAM OF THE BEST MODEL BEFORE AND AFTER TEMPERATURE SCALING	43
A2.1	PUBLIC LANDING PAGE (LEFT) AND SIGN-IN PAGE (RIGHT)	52
A2.2	PLAYER DASHBOARD WITH SCOREBOARD, TODAY'S PLAN, BATTING PROFILE AND BEST PHOTO	52
A2.3	PHOTO POSTURE REPORT: QUALITY SCORE, BODY MAP WITH MEASURED ANGLES AND DETECTION QUALITY	53
A2.4	VIDEO SHOT ANALYSIS: HEATMAP, ATTENTION TIMELINE AND PREDICTED SHOT WITH CALIBRATED CONFIDENCE	53
A2.5	COMPARISON WITH A REFERENCE CLIP: DTW DISTANCE AND PER-JOINT DIFFERENCES	54
A2.6	AI COACH EXPLAINING A COVER DRIVE WITH MEASURED FACTS AND GENERAL TIPS	54
A2.7	HISTORY OF ANALYSES (LEFT) AND THE ONE-CLICK PDF REPORT OF A SHOT ANALYSIS (RIGHT)	55
A2.8	ADMINISTRATOR OVERVIEW (LEFT) AND MODEL RESULTS (RIGHT)	55

LIST OF TABLES

TABLE NUMBER	TABLE	PAGE NUMBER
2.1	RESULTS REPORTED FOR THE EXISTING SYSTEM [1]	10
2.2	COMPARISON OF CRICKET SHOT CLASSIFICATION STUDIES	11
3.1	INNOVATIONS OF THE PROPOSED SYSTEM OVER [1]	17
4.1	HARDWARE REQUIREMENTS	18
4.2	SOFTWARE REQUIREMENTS	19
5.1	DATABASE TABLES AND THEIR MAIN COLUMNS	25
6.1	JOINT ANGLES COMPUTED FROM MEDIAPIPE LANDMARKS	28
6.2	TRAINING CONFIGURATION	30
6.3	WEAK-SPOT RULES USED TO CHOOSE DRILLS	33
6.4	LEVELS AND THEIR REQUIREMENTS	33
7.1	UNIT TEST CASES	36
7.2	INTEGRATION TEST CASES	37
7.3	SYSTEM TEST CASES (FR AND NFR REFER TO THE FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS OF THE PROJECT SPECIFICATION)	38
7.4	CLASSIFICATION RESULTS ON THE CRICSHOT10K TEST SPLIT (1,289 CLIPS)	40
7.5	MCNEMAR EXACT TEST BETWEEN MODELS ON THE SAME TEST CLIPS	41
7.6	CALIBRATION BEFORE AND AFTER TEMPERATURE SCALING	42
7.7	MACRO-F1 (%) BY CAMERA VIEW AND IMAGE SHARPNESS	44
9.1	WORK SCHEDULE FOR PHASE II	49

LIST OF ABBREVIATIONS

S.NO	ABBREVIATIONS	DESCRIPTION
1	**AI**	Artificial Intelligence
2	**API**	Application Programming Interface
3	**CNN**	Convolutional Neural Network
4	**CPU**	Central Processing Unit
5	**CSS**	Cascading Style Sheets
6	**DFD**	Data Flow Diagram
7	**DTW**	Dynamic Time Warping
8	**ECE**	Expected Calibration Error
9	**ER**	Entity Relationship
10	**FR**	Functional Requirement
11	**GPU**	Graphics Processing Unit
12	**Grad-CAM**	Gradient-weighted Class Activation Mapping
13	**GRU**	Gated Recurrent Unit
14	**HTML**	HyperText Markup Language
15	**JSON**	JavaScript Object Notation
16	**JWT**	JSON Web Token
17	**LLM**	Large Language Model
18	**LRCN**	Long-term Recurrent Convolutional Network
19	**LSTM**	Long Short-Term Memory
20	**MLP**	Multi-Layer Perceptron
21	**NFR**	Non-Functional Requirement
22	**PDF**	Portable Document Format
23	**REST**	Representational State Transfer
24	**SQL**	Structured Query Language
25	**UML**	Unified Modeling Language
26	**XP**	Experience Points
27	**YOLO**	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Watch any cricket match on television and you will see the same short event repeated hundreds of times. The bowler runs in, the ball is delivered, and in roughly one second the batter decides on a stroke and plays it. That one-second window is where most of the interesting information in a match sits. A coach wants to know which strokes a young player leans on and which ones break down under pressure. An analyst preparing for an opponent wants to know where a batter scores and how they get out. A broadcaster wants to pull every cover drive of an innings into a highlight package before the next over begins.

All of these needs depend on a simple label: what shot was played? Surprisingly, that label is still mostly produced by a person sitting in front of a screen. The analyst pauses the video after each ball, decides on the shot and types it into a spreadsheet or a tagging tool. It is tiring work, it takes at least as long as the match itself, and it is less objective than it looks. A pull and a hook, or a square cut and a late cut, can look almost the same, and two experienced people will not always agree. At school and club level, where there is usually no analyst at all, a player's feedback is whatever the coach happened to notice from the side of the net.

Computer vision has matured to a point where much of this can be automated. Convolutional neural networks recognise people and objects reliably, recurrent networks can follow how a movement develops across frames, and pose estimators such as MediaPipe can locate the joints of a human body in a photograph in a few milliseconds on an ordinary laptop. These tools are already used to track players in football and basketball, to judge baseball swings and to classify tennis strokes [20]-[23].

What held cricket back was data. Earlier cricket shot datasets were small, covered only a handful of shots, or had problems with duplicated and mislabelled clips. In 2026, Dihan et al. published CricShot10k [1], a collection of 10,086 one-second broadcast clips

covering 15 different batting shots, labelled independently by three annotators who agreed very strongly with each other. Alongside the data they released CricShotNet, a model that combines an EfficientNetV2-S image network with a small recurrent layer and identifies the correct shot about 89% of the time. That work is the starting point of this project.

1.2 BATTING STROKES IN BRIEF

Because this report is written for readers in computer science rather than cricket, it helps to describe what the system is trying to recognise. A batting stroke can be understood through three questions: which foot carries the weight, whether the bat comes down straight or across the line of the ball, and whether the ball is kept along the ground or hit in the air.

- Front-foot strokes are played by stepping towards the ball. The cover drive, straight drive, flick and forward defence belong to this group. Fig. 1.1 shows a batter moving onto the front foot to meet the ball.
- Back-foot strokes are played by moving back towards the stumps to a shorter ball. The square cut, late cut, pull and hook fall here.
- **Straight-bat versus cross-bat.** Drives and defensive strokes keep the bat vertical; cuts, pulls, hooks and sweeps swing the bat horizontally across the line of the ball.
- Improvised strokes such as the reverse sweep, scoop and upper cut are newer, mostly seen in limited-overs cricket, and deliberately break the classical rules.



Fig: 1.1 A batter moving onto the front foot to meet the ball

Two practical points follow from this. First, many shots share the same starting stance and differ only in the last third of a second, so a model has to look at the sequence of frames rather than at a single picture. Second, the body tells a lot of the story: how far the front knee bends, where the head is relative to the front foot, and whether the shoulders open or stay side-on. This is why the project combines whole-frame video features with body-pose measurements.

1.3 MOTIVATION

The project grew out of three observations made while studying CricShot10k and earlier work.

A label alone does not help a player improve. If a system says "cover drive, 91%", a broadcaster has what they need, but a player does not. The player wants to know what their body did, what a good version of the stroke looks like, and what to practise. A coach, before trusting the system, wants to see that it is looking at the bat and the batter and not at the scoreboard. None of the published cricket shot systems provide that kind of evidence.

Research code is not something a coach can use. CricShot10k is released as data and training scripts. Someone who wants to analyse their own net session needs to install deep learning libraries, convert their video and run a script from the command line. There is no upload page, no history of past sessions and no report to share. Body pose looks useful, but nobody has measured it on this dataset. Pose landmarks are not affected by jersey colour or lighting, which are two of the biggest nuisances in broadcast footage. On the other hand, a skeleton throws away the bat and the ball, and an earlier pose-only study on a smaller dataset [6] reported noticeably lower accuracy than video-based models. Whether pose adds anything on top of a strong video model is an open question, and it is the main research question of this project.

1.4 PROBLEM STATEMENT

The problem addressed in this project can be stated as follows. Given either a single photograph of a batter or a short video clip of one batting stroke, build a system that: 1. reports the batter's posture in measurable terms (joint angles with visibility) from a photograph, without guessing at joints it cannot see;

1. classifies the stroke in a video clip into one of the 15 CricShot10k classes, with a confidence value that reflects the real chance of being correct;
2. shows visual and temporal evidence for each classification;
3. compares the player's joint movement with a reference clip of the same stroke, even when the two were played at different speeds;
4. presents all of this through a web application and a conversational coach that explains the results in plain language without inventing facts.

In addition, the classifier must be evaluated fairly against the published baseline, and the effect of adding pose features must be measured on exactly the same data split.

1.5 OBJECTIVES

The objectives of the project are listed below. Each one is carried through to one or more modules in Chapters 5 and 6 and to one or more experiments in Section 7.6.

1. Rebuild the baseline. Re-implement the CricShotNet video model in PyTorch and train it on CricShot10k, so that every later comparison starts from a known, fair reference point.
2. Add body pose. Extract MediaPipe pose landmarks from photographs and from each sampled video frame, and turn them into joint angles and normalised coordinates.
3. Compare five models. Train video-only, pose-only (2D and 3D) and combined video + pose (2D and 3D) classifiers on the same split and compare them on accuracy, macro-F1, precision, recall, calibration and robustness to the camera view.
4. Offer two analysis modes. Provide photo analysis and video analysis in one web application with an HTML, CSS and JavaScript front end and a FastAPI back end.
5. Explain and coach. Produce Grad-CAM heatmaps, an attention-based key-frame timeline and a DTW comparison with a reference clip, and add an AI coach that explains stored results in everyday language and an adaptive training coach that plans daily practice from the player 's own measurements and tracks progress over time.
6. Make confidence honest. Calibrate the classifier with temperature scaling and flag low-confidence clips as uncertain instead of guessing.

1.6 SCOPE OF THE PROJECT

Included: batting strokes by right- and left-handed batters in broadcast-style footage; the 15 shot classes of CricShot10k; single-person photographs for posture reports;

English-language coaching replies; deployment on a single server for academic evaluation.

Excluded: bowling and fielding; ball tracking and trajectory prediction; live streams; strokes outside the 15 classes such as the leave, leg glance and back-foot punch; medical, injury or biomechanical diagnosis; languages other than English.

One design decision deserves to be stated up front. Photo mode never predicts a shot type. A single still image cannot show how the bat is moving, and the literature shows that single-frame shot classification depends on a person choosing the right frame [1]. The photograph is therefore used only for what it can honestly support: a posture report.

1.7 ORGANISATION OF THE REPORT

Chapter 2 reviews the literature, describes the existing CricShotNet system and lists its disadvantages. Chapter 3 presents the proposed system and the architecture of the proposed method. Chapter 4 gives the hardware and software requirements and describes the software used. Chapter 5 presents the system design through data flow, use case, class, sequence, entity relationship and collaboration diagrams. Chapter 6 describes the five modules of the system. Chapter 7 covers testing, including the evaluation of the shot classifiers on the CricShot10k test split. Chapter 8 lists the advantages of the proposed system, and Chapter 9 concludes the report with the work planned for Phase II. Appendix 1 gives sample code and Appendix 2 shows screenshots of the working application.

CHAPTER 2

LITERATURE SURVEY

A literature survey for this project has to cover more ground than a typical classification paper, because SmartCricket AI touches several fields at once: sports video analysis, action recognition, human pose estimation, model explainability, confidence calibration and language-based feedback. This chapter walks through each area, notes what each piece of work contributes and where it stops short, and ends with a comparison table and a list of the gaps that shaped the design of the project.

2.1 EXISTING SYSTEM

In practice the existing system for most teams is manual. An analyst watches each delivery and records the shot in a spreadsheet or a commercial video-tagging tool. The analyst may also clip the delivery and attach notes. This process takes roughly as long as the footage itself, often longer, and depends on the experience of the person tagging. At academy level, coaches observe players live and give verbal feedback, with no stored record that a player can review later.

2.1.1 Recognising shots from single images

Several groups have tried to recognise batting shots from still images. Foysal et al. [13] trained a convolutional network named Shot-Net on frames of batters. Fernandes et al. [14] followed a similar route with a 2D CNN. Suryavanshi et al. [15] extracted CNN features and classified them with a support vector machine, while Dey and Biswas [16] applied a vision transformer to the same task. An older approach by Karmaker et al. [17] used motion vectors between frames, which works for a fixed camera but breaks down in broadcast footage where the camera pans to follow play. The common weakness of the single-image approach is easy to see once the primer in Section 1.2 is kept in mind. Many shots start from the same stance and only become distinguishable as the bat comes through, so a single image either misses the decisive moment or requires a person to pick it out by hand. In other words, single-image methods move the hard part of the problem back to a human. This observation directly shaped our image mode: we keep photographs

because club players often have nothing else, but we use them only for posture measurements and never for a shot label.

2.1.2 Recognising shots from video

Sen et al. [12] built a larger set of about 1,900 clips over ten shots and reported 93% accuracy with a VGG16 network followed by a GRU. On paper this is the highest accuracy in the area. Looking more closely, however, some clips are several seconds long, which means the network sees much more than the stroke itself. A share of the clips are mirrored copies of others, which inflates the dataset size and, if copies end up on both sides of the train/test split, can also inflate the accuracy [1]. The lesson for our project is practical: augmentation such as flipping must be applied only after the split, and the split must be checked for overlapping source videos.

2.1.3 CricShot10k and the CricShotNet pipeline

The CricShot10k work by Dihan et al. [1] is the foundation of this project, so it is described in more detail. The authors started from 536 highlight videos totalling about 107 hours. Instead of cutting clips by hand, they used object detectors to find moments where the batter, the ball and the bowler were all on screen and cut a one-second clip at each such moment. This produced 14,369 candidate clips, of which 10,086 were kept after three annotators labelled them into 15 shot classes. All three annotators agreed on 9,218 clips, and their overall agreement, measured with Fleiss' kappa [36], was 0.9376, which is very high.

For classification, the authors tried 15 image backbones and 9 recurrent variants and found that EfficientNetV2-S followed by a 128-unit GRU worked best. Two preprocessing steps added a few points of accuracy: cropping each frame around the batter, and tinting the batter and bat using a segmentation model so that the network pays less attention to clothing colour (Fig. 2.1). Their best model was right 89% of the time on the first guess and 99% of the time within its top three guesses. The pull and the hook were the pair most often confused, which matches the intuition from Section 1.2, since both are cross-bat strokes to short balls. A lighter MobileNetV3 version reached 84%.

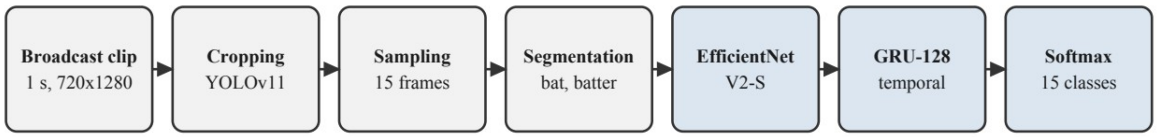
What CricShotNet does not do is equally important for this project. It returns a label and a probability, with no indication of what in the frames supported the decision and no check on whether the probability is trustworthy. It was evaluated on broadcast footage only. And it is a research artefact rather than a tool a coach or player can use.



Fig: 2.1 Preprocessing in CricShotNet: cropped frame (left) and the same frame with the bat and batter segmented (right)

The automated state of the art for cricket shot classification is CricShotNet [1]. Its pipeline, shown in Fig. 2.2, works as follows.

1. A one-second broadcast clip at 720 x 1280 pixels is taken as input.
2. YOLOv11 [29] detects the batter and the bat, and the frames are cropped around them.
3. Fifteen frames are sampled evenly from the clip and resized to 224 x 224 pixels.
4. A segmentation step tints the batter and the bat so that jersey colour and lighting matter less.
5. EfficientNetV2-S [3], pre-trained on ImageNet, extracts a feature vector from each frame.
6. A GRU with 128 units processes the 15 feature vectors in order and follows the swing over time.
7. A dense layer with softmax outputs a probability for each of the 15 shot classes.



Reported: top-1 89%, top-2 97%, top-3 99% on the CricShot10k test split [1]

Cropping + segmentation 89.09% vs cropping alone 85.76% [1]

Fig: 2.2 CricShotNet pipeline of the existing system (from [1])

The authors chose this configuration after comparing 15 CNN backbones and 9 recurrent variants. Horizontal flipping is applied to training clips only, to avoid leakage between splits, and training uses the Adam optimiser [38] with a learning rate of 10 and early stopping. The reported accuracies are summarised in Table 2.1.

Table 2.1 Results reported for the existing system [1]

Measure	Value
Top-1 accuracy, full CricShotNet	89%
Top-2 accuracy	97%
Top-3 accuracy	99%
Cropping + segmentation	89.09%
Cropping only	85.76%
MobileNetV3 variant, top-1	84%
Most confused pair	Pull vs hook (13%)
Inter-annotator agreement (Fleiss' kappa)	0.9376

2.1.4 Pose estimation

OpenPose [33] made real-time multi-person pose estimation widely available by predicting body parts and the connections between them. BlazePose [19], which powers MediaPipe Pose [43], focuses on one person at a time and returns 33 landmarks, each with a visibility score, fast enough to run on a phone. Graph-based models such as ST-GCN [34] treat a skeleton sequence as a graph that changes over time and are strong at recognising actions from skeletons alone.

MediaPipe is the right fit for this project for three reasons. It runs on a CPU, so the photo mode needs no GPU. It gives a visibility score for every landmark, which lets us refuse to report an angle for a joint that is hidden behind the body or the bat. And it offers an

image mode and a video mode with the same landmark definitions, so a posture report from a photo and an angle sequence from a clip are directly comparable.

To compare a player's movement with a reference movement, the two sequences must first be lined up in time, because no two players swing at exactly the same speed. Dynamic Time Warping [10], originally developed for matching spoken words, finds the cheapest way to stretch and compress one sequence to match another, and it remains a standard tool for comparing gestures and motions.

2.1.5 Comparison of related work

Table 2.2 brings the most relevant shot classification studies together. Each accuracy figure comes from a different dataset with a different number of classes, so the numbers should not be compared directly; the table is meant to show coverage and limitations rather than to rank the methods.

Table 2.2 Comparison of cricket shot classification studies

Study	Data	Method	Accuracy	Main limitation
Foysal et al. 2019 [13]	Still images	CNN (Shot-Net)	-	One frame cannot show bat movement
Fernandes et al. 2023 [14]	Still images	2D CNN	-	One frame; few classes
Dey and Biswas 2024 [16]	Still images	Vision transformer	-	Key frame chosen by a person
Hoque et al. 2023 [11]	about 1,300 clips, 4 shots	LRCN	73%	Test cricket only, right-handers only
Sen et al. 2021 [12]	about 1,900 clips, 10 shots	VGG16 + GRU	93%	Long clips, mirrored copies counted
Rao et al. 2025 [6]	about 1,900 clips, 6 shots	MediaPipe pose	80%	No bat, ball or context in skeleton
Dihan et al. 2026 [1]	10,086 clips, 15 shots	EfficientNetV2-S + GRU	89%	No explanation, no calibration, no app

2.2 DISADVANTAGES

The points below are gaps with respect to the goals of this project. They are not failures shown in the experiments of [1]; where the paper itself supports a point, it is cited.

- **Fixed set of 15 shots.** The leave, leg glance and back-foot punch are not covered; the authors list them as future work [1].
- **Depends on picture conditions.** Camera angle, lighting, jersey colour and occlusion vary across clips [1]. Phone or academy video, which is filmed very differently from broadcast footage, has not been tested.
- **No explanation.** The output is a label and a probability. Nothing shows which part of the frame or which moment of the swing drove the answer, so a coach cannot check whether the model is right for the right reason.
- **Uncalibrated confidence.** The softmax probability is used directly. Deep networks are often over-confident [9], so a reported 0.9 may not mean that the prediction is right nine times out of ten.
- **No application or feedback.** The work is released as data and research code. There is no upload interface, no stored history, no comparison with a model player and no coaching feedback.
- **Pose is unproven here.** Pose alone loses texture and context [1], [6]; whether it helps when combined with the video model has not been tested, and it adds compute that must be justified.
- **No support for photographs.** Many club players only have a photograph of their stance or follow-through, which the existing system cannot use.

Reading the work above side by side, six gaps stand out. Each one maps to a feature of SmartCricket AI.

1. No explanations. None of the cricket shot systems shows why it made a prediction. SmartCricket AI adds Grad-CAM heatmaps and a key-frame timeline.
2. Untested value of pose. Pose has been used alone, but not added to a strong video model on a large dataset. Experiment E3 tests exactly this.
3. Untrustworthy confidence. Reported probabilities are uncalibrated. SmartCricket AI applies temperature scaling and flags uncertain results.

4. No comparison with good technique. No system lines up a player's movement with a reference performance. SmartCricket AI uses DTW on joint angles.
5. No usable product. All prior work stops at a dataset and a model. SmartCricket AI is a working web application with history, reports and a coach.
6. Misuse of single images. Image-based methods try to read a shot from one frame. SmartCricket AI uses a photo only for what it can support: a posture report.

CHAPTER 3

PROPOSED SYSTEM

SmartCricket AI keeps the strengths of CricShotNet and adds the missing parts around it. It offers two ways to analyse a batter, shown in Fig. 3.1, and shares one platform, one database and one AI coach between them.

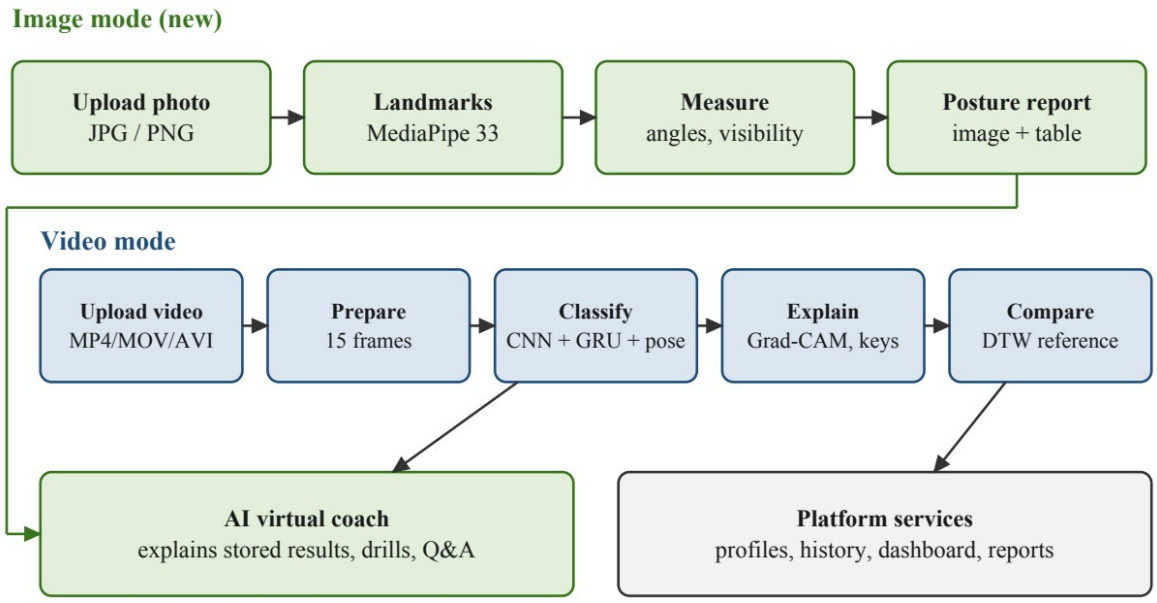


Fig: 3.1 Overview of the proposed SmartCricket AI system

3.1 ARCHITECTURE OF THE PROPOSED METHOD

SmartCricket AI follows a layered client-server architecture, shown in Fig. 3.2. The browser talks only to the API layer. The API layer validates requests and passes media to the preprocessing layer, which prepares inputs for the two inference paths: the shot classifier and the pose analysis path. Their outputs are merged by the results service, stored in PostgreSQL and file storage, and passed to the AI coach on request.

The layers have the following responsibilities.

- **Presentation layer.** Static HTML, CSS and JavaScript pages for the dashboard, image mode, video mode, coach and history. Calls the REST API with Fetch and renders JSON results with Chart.js.

- **API layer.** FastAPI routers for authentication, image analysis, video analysis, references, coach, history and reports. Handles token checks, request validation and error responses.
- **Preprocessing layer.** OpenCV routines for decoding, sampling, resizing and normalising. The same code is used during training and inference so that the model sees identical inputs.
- **Inference layer.** The PyTorch classifier with Grad-CAM and attention, and the MediaPipe pose path with angle calculation and DTW.
- **Results and coach.** Merges outputs into one result object, applies calibration and the uncertainty flag, and builds grounded prompts for the LLM.
- **Storage.** PostgreSQL for structured records; the file system for uploads, heatmaps and reports.

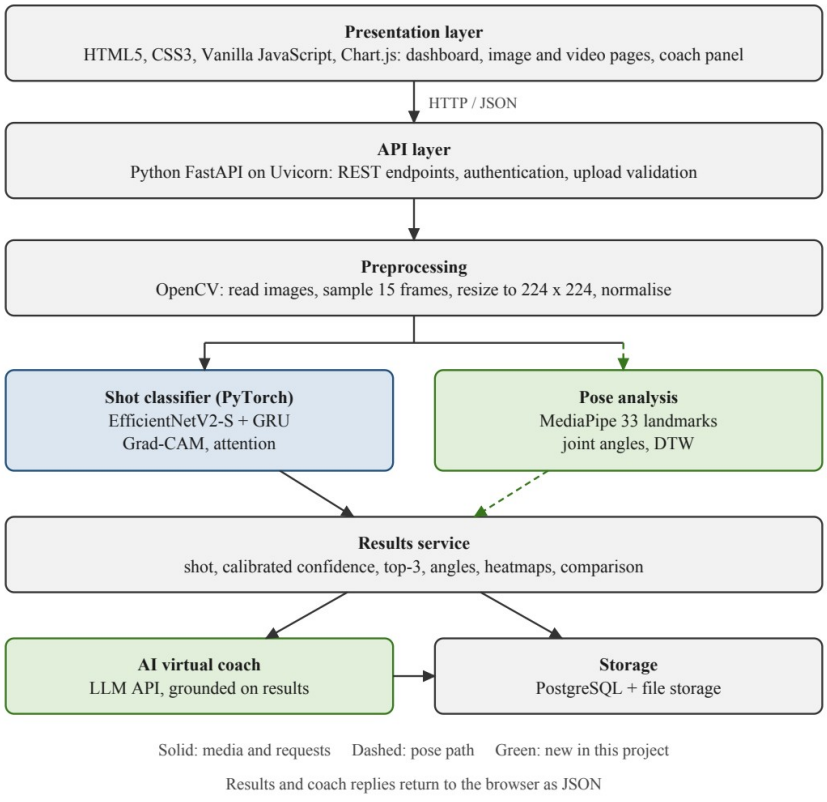


Fig: 3.2 Layered architecture of SmartCricket AI

3.2 IMAGE MODE

The user uploads a JPG or PNG photograph of a batter. MediaPipe Pose Landmarker [43] in image mode detects up to 33 body landmarks with visibility scores. The system

computes front-knee, front-elbow, shoulder-line and hip angles from clearly visible points only and returns an annotated image drawn with OpenCV and a table of measurements. No shot type is predicted from a photograph, because one frame cannot show the movement of the bat [1].

3.3 VIDEO MODE

The user uploads a short MP4, MOV or AVI clip. OpenCV samples 15 evenly spaced frames and resizes them to 224 x 224. Several classifiers are available, of which the three main ones are: the video-only model that re-implements CricShotNet, a pose-only model fed with the joint-angle sequence, and a fused video + pose model. The prediction is calibrated with temperature scaling, and clips below a confidence threshold are marked uncertain. Grad-CAM heatmaps show where the model looked in each frame, an attent ion layer over the GRU outputs ranks the frames into a key-frame timeline, and DTW aligns the player's joint-angle sequence with a reference clip of the same shot.

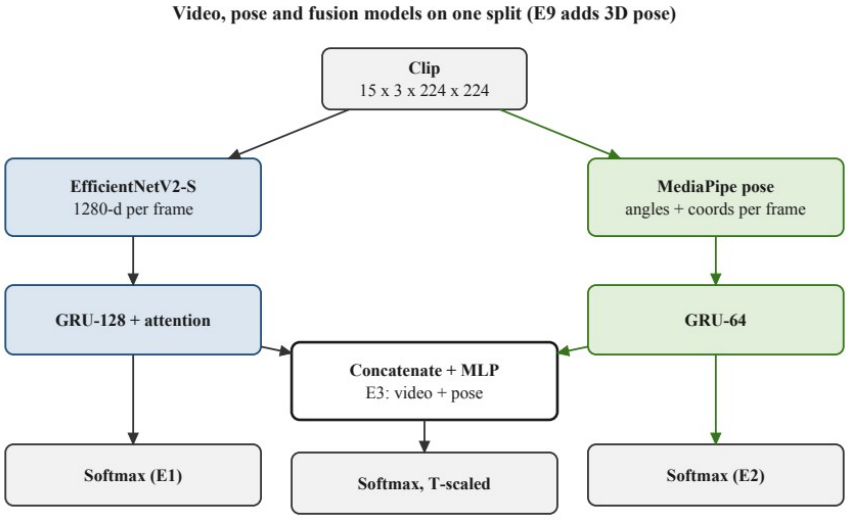


Fig: 3.3 Classifier variants compared in experiments E1 to E3; E9 repeats the pose and fusion models with 3D pose

3.4 AI VIRTUAL COACH AND PLATFORM

A large language model, called from the server, receives only the stored results of the current session and explains them in plain words. Its reply separates "Measured" facts taken from the analysis from "General tip" coaching knowledge, suggests practice drills and answers follow-up questions. The platform adds player accounts, session history, a

dashboard of shot mix and confidence trends, a model comparison page and downloadable CSV and HTML reports.

3.5 KEY INNOVATIONS

Table 3.1 lists what is new compared with the existing system.

Table 3.1 Innovations of the proposed system over [1]

No.	Feature	Description
1	Image analysis mode	Posture report from a single photograph: landmarks, angles and visibility.
2	Three-way comparison	Video-only, pose-only and video + pose trained and tested on the same split.
3	Grad-CAM heatmaps	Shows which parts of the bat and body led to the prediction in each frame.
4	Key-frame timeline	Attention weights show which of the 15 frames mattered most.
5	Reference comparison	Aligns the player's joint angles with a reference clip using DTW.
6	AI virtual coach	Explains results, suggests drills and answers follow-up questions, grounded in stored results.
7	Honest confidence	Temperature scaling so that a stated 80% means about 80%; unsure clips are flagged.
8	Player profiles	Session history, shot mix, trends and downloadable reports.

CHAPTER 4

SYSTEM REQUIREMENTS

This chapter lists the hardware and software needed to train the models and to run the SmartCricket AI application, and describes the main software tools used in the project.

4.1 HARDWARE REQUIREMENTS

The web application runs on an ordinary laptop without a graphics card; all measurements in Chapter 7 were taken on an Apple M4 laptop CPU. Training the shot classifiers needs a GPU, for which the free NVIDIA T4 GPU of Google Colab was used. Table 4.1 gives the minimum and recommended hardware.

Table 4.1 Hardware requirements

Component	Minimum	Recommended
Processor	Intel Core i5 (8th gen) or equivalent	Intel Core i7 / AMD Ryzen 7
Memory (RAM)	8 GB	16 GB or more
GPU (training)	NVIDIA GPU with 6 GB VRAM	NVIDIA RTX 3060 (12 GB) or better
Storage	50 GB free	256 GB SSD
Display	1366 x 768	1920 x 1080
Client device	Any device with a modern browser	Laptop or tablet

4.2 SOFTWARE REQUIREMENTS

Table 4.2 lists the software used. All of it is free and open source, except the hosted language model, which is called through its public API.

Table 4.2 Software requirements

Category	Software
Operating system	Ubuntu 22.04 LTS, Windows 10/11 or macOS 13+
Language	Python 3.10+, JavaScript (ES2020)
Deep learning	PyTorch 2.x, torchvision
Computer vision	OpenCV 4.x, MediaPipe 0.10+
Scientific computing	NumPy, scikit-learn
Web framework	FastAPI, Uvicorn, Pydantic
Database	PostgreSQL 15+, SQLAlchemy, psycopg
Front end	HTML5, CSS3, vanilla JavaScript, Chart.js
Language model	Any hosted LLM API with a chat endpoint
Development tools	VS Code, Git and GitHub, Python venv, Jupyter
Browser	Chrome, Edge, Firefox or Safari (current versions)

4.3 SOFTWARE DESCRIPTION

4.3.1 Python

Python 3 is used for the back end, the vision pipeline and the training notebook, with NumPy, scikit-learn and pandas for arrays, metrics and result tables.

4.3.2 PyTorch and torchvision

PyTorch is the deep learning framework used to define and train the GRU and attention based shot classifiers, the fusion network and the temperature scaler. torchvision supplies the ImageNet pretrained backbones (EfficientNetV2-S, and RegNetY-16GF in the improved models) whose last convolutional block is also used for Grad-CAM heatmaps.

4.3.3 MediaPipe and OpenCV

MediaPipe Pose Landmarker detects 33 body landmarks in 2D and 3D in a few milliseconds per frame on a CPU. OpenCV reads video files, samples frames, resizes and crops them, draws the skeleton and heatmap overlays and measures image sharpness and brightness for the input quality check.

4.3.4 FastAPI and Uvicorn

FastAPI is a modern Python web framework that builds REST endpoints from type annotated functions and validates every request with Pydantic. Uvicorn serves it. Heavy video analysis runs in a worker thread so that the server keeps answering other requests.

4.3.5 PostgreSQL and SQLAlchemy

PostgreSQL stores users, uploads, analyses, coach messages, training plans and model results. JSONB columns keep per-frame angles, top-3 lists and summaries. SQLAlchemy maps the tables to Python classes and psycopg connects to the database.

4.3.6 HTML5, CSS3, JavaScript and Chart.js

The front end is written in plain HTML5, CSS3 and JavaScript without a framework, so every page loads quickly on phones. Chart.js draws the angle timelines, progress charts and model result charts, and MediaPipe for JavaScript powers the live mirror inside the browser.

4.3.7 Google Colab

Google Colab provides a free NVIDIA T4 GPU. The training notebook caches pose and video features on Google Drive, so an interrupted run continues where it stopped.

4.3.8 Large language model API

A hosted large language model (OpenAI or Gemini, chosen in the settings) powers the virtual coach. It receives only the numbers stored for an analysis and a fixed instruction to separate measured facts from general tips, and every number in its reply is checked against the stored data.

CHAPTER 5

SYSTEM DESIGN

System design turns the requirements of Chapter 4 into a structure that can be implemented. This chapter presents the data flow diagrams, the use case, class and sequence diagrams, the entity relationship diagram of the database and the collaboration diagram of a video analysis request.

5.1 DATA FLOW DIAGRAM

5.1.1 Level 0 data flow diagram

The context diagram (Fig. 5.1) treats SmartCricket AI as a single process exchanging data with three external entities: the player or coach, the administrator and the LLM service.

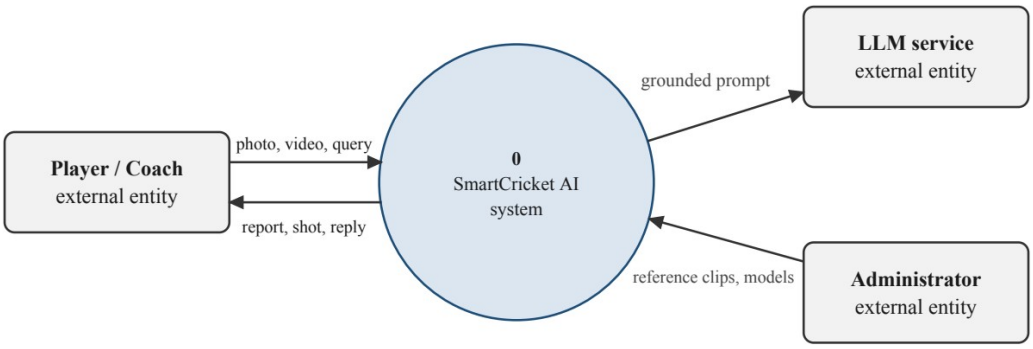


Fig: 5.1 Level 0 data flow diagram (context diagram)

5.1.2 Level 1 data flow diagram

The level 1 diagram (Fig. 5.2) splits the system into five processes: authentication, upload validation, image analysis, video analysis, and coaching and reporting, with four data stores.

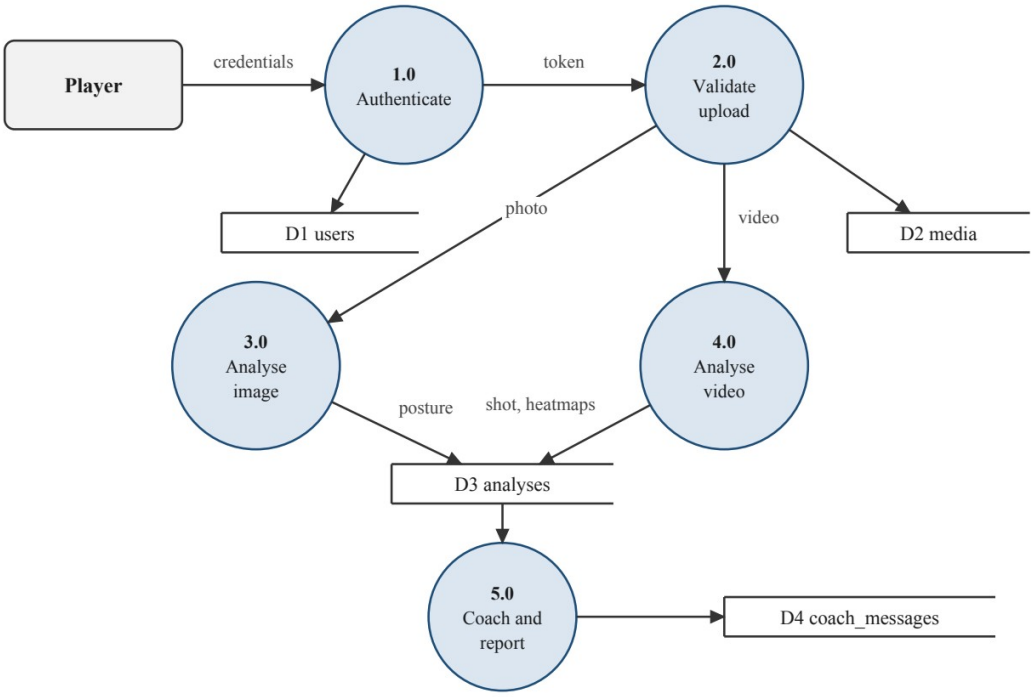


Fig: 5.2 Level 1 data flow diagram

5.2 USE CASE DIAGRAM

Fig. 5.3 shows the actors and their use cases. The player or coach performs all analysis use cases; the administrator manages reference clips, users and models; the LLM service participates in the coach use case.

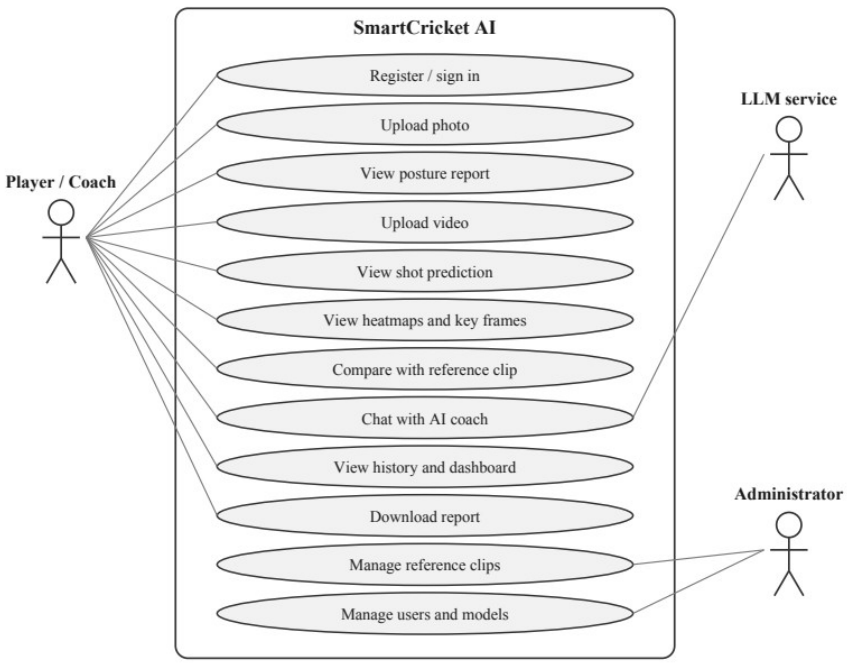


Fig: 5.3 Use case diagram

5.3 CLASS DIAGRAM

Fig. 5.4 shows the main classes. User, Media and Analysis are persistent entities. VideoPreprocessor, ShotClassifier and PoseEstimator implement the analysis pipeline, and CoachService reads stored analyses.

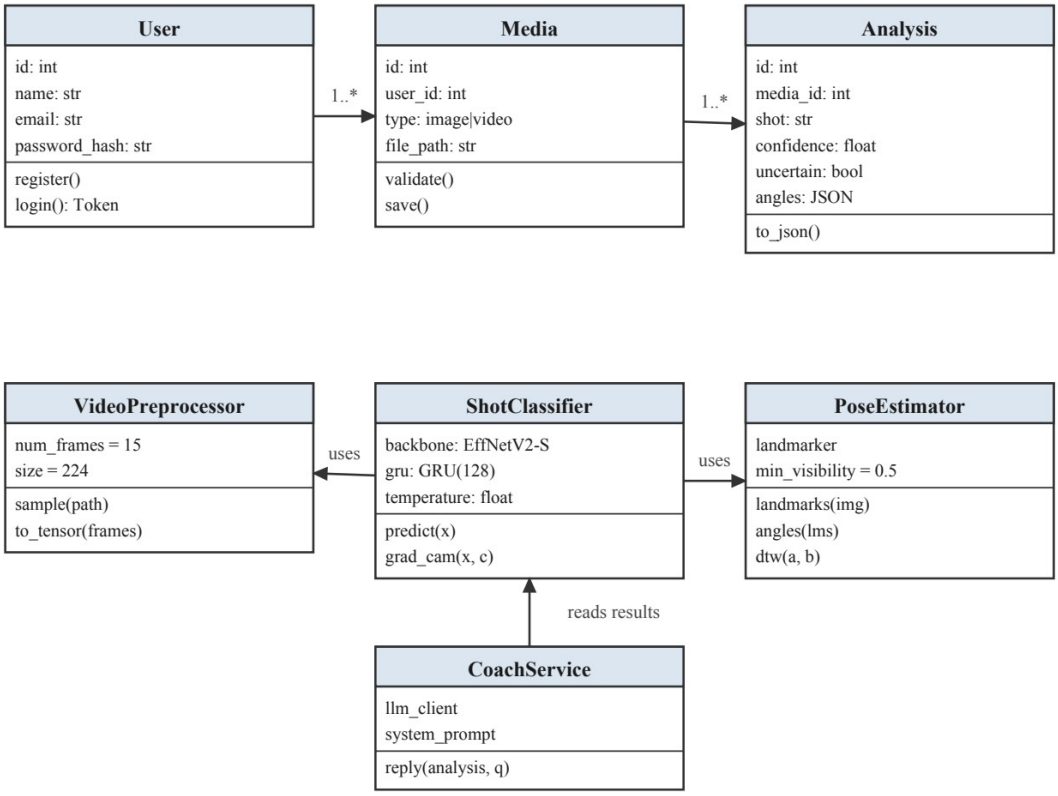


Fig: 5.4 Class diagram

5.4 SEQUENCE DIAGRAM

Fig. 5.5 shows the sequence of messages for a video analysis request, from upload to the JSON response.

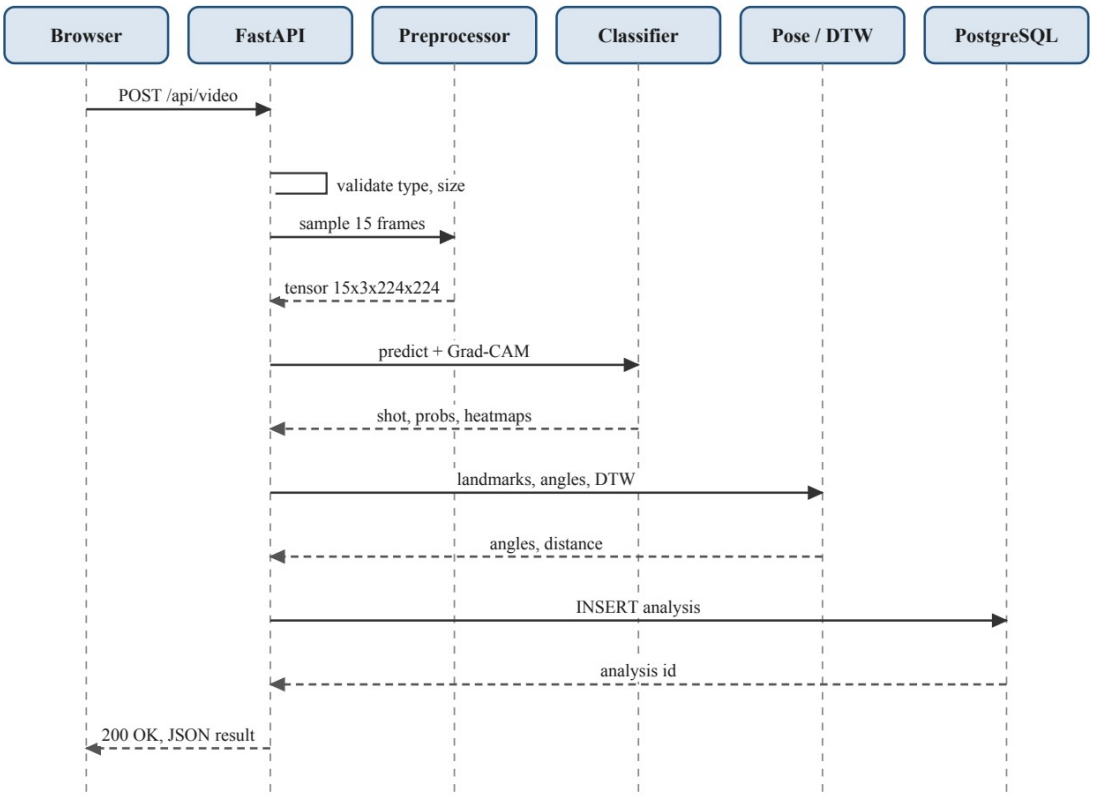


Fig: 5.5 Sequence diagram for video analysis

5.5 ENTITY RELATIONSHIP DIAGRAM

The database has twelve tables in PostgreSQL. The six core tables and their relationships are shown in Fig. 5.6; Table 5.1 lists every table with its main columns. All tables use an integer primary key, every player-owned row references users.id and is deleted with its user, and JSON columns (JSONB in PostgreSQL) hold per-frame results that do not need separate querying.

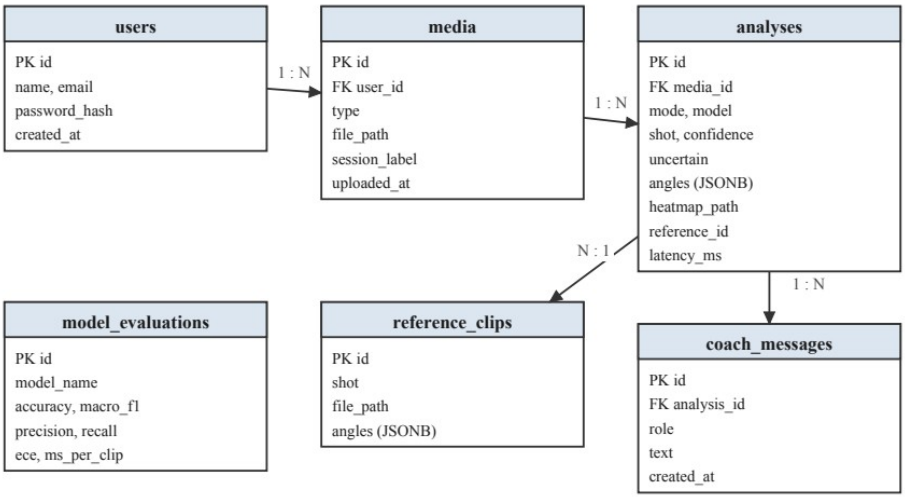


Fig: 5.6 Entity relationship diagram

Table 5.1 Database tables and their main columns

Table	Main columns	Purpose
users	name, email (unique), password_hash (bcrypt), role, created_at	Accounts; role is player or admin
media	user_id, type, file_path, original_name, session_label, uploaded_at	Uploaded photos and videos
analyses	media_id, mode, model, shot, confidence, uncertain, top3, angles, attention, summary (JSON), dtw, latency_ms	One result per analysis
coach_messages	analysis_id, role, text, created_at	Conversation about one result
reference_clips	shot, file_path, angles (JSON), added_by	Model-player clips for comparison
model_evaluations	model_name, accuracy, macro_f1, precision, recall, ece_before, ece_after	Imported test results
daily_logs	user_id, day (unique together), balls_faced, soreness, feeling	Morning check-in
plan_drills	user_id, day, code, title, dose, reason, difficulty, done	The daily drill plan
quiz_answers	user_id, day (unique together), question, choice, correct	Question of the day
reflections	user_id, analysis_id (unique), question, answer	Answers to the coach's question about an upload
weekly_reports	user_id, week_start (unique together), text, stats	Worded weekly summary
training_chats, live_sessions	user_id, role / text; title, duration_s, reps, hold_s, stats	Training chat and saved live sessions

5.6 COLLABORATION DIAGRAM

The collaboration diagram in Fig. 5.7 shows the same video analysis request as the sequence diagram, but organised around the objects that work together instead of around time. Each link carries the numbered messages exchanged between two objects. The browser posts a clip to the FastAPI video router (1), which checks the file (2) and hands it to the VideoService (3). The service samples 15 frames and asks the MediaPipe pose landmarker for the batter's landmarks (4 to 6), crops the frames around the batter and passes them with the pose features to the fusion shot classifier, which returns the class scores and attention weights and the Grad-CAM heatmaps (7 to 9). The router stores the media and analysis rows in PostgreSQL (10) and returns the result to the browser (11, 12). When the player then asks the coach a question (13), the coach service reads only the stored numbers, saves the conversation (14, 15) and returns a grounded reply (16).

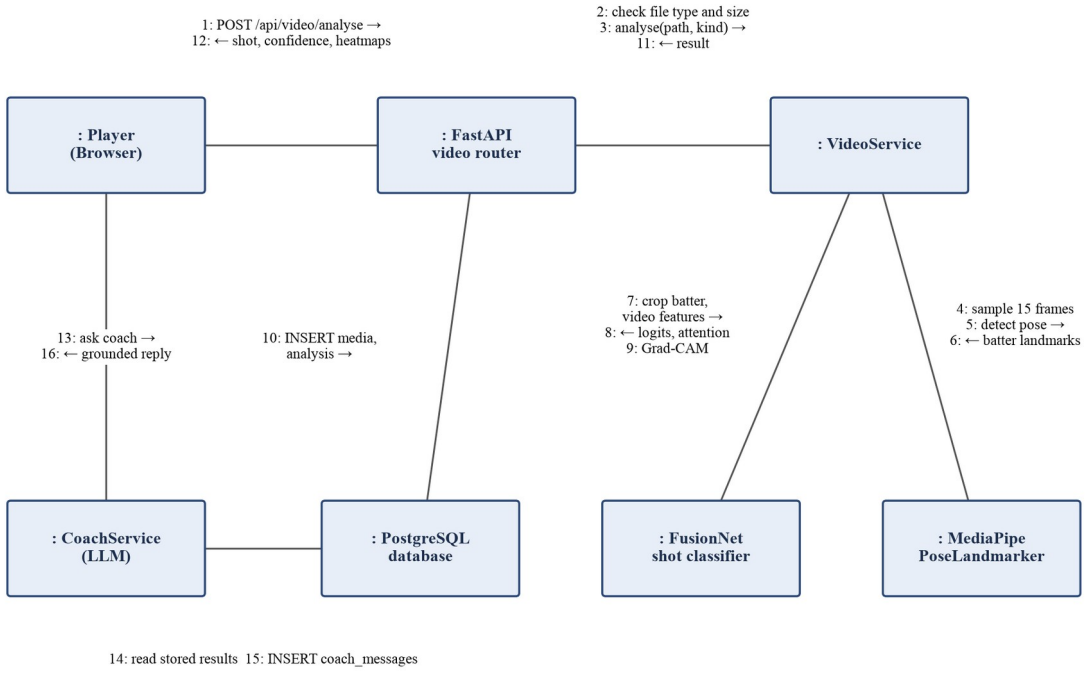


Fig: 5.7 Collaboration diagram for a video analysis and a coach question

CHAPTER 6

MODULE DESCRIPTION

SmartCricket AI is built from nine design modules (M1 to M9). For this report they are grouped into five functional modules: user management and administration (M1, M8), photo posture analysis (M2), video preprocessing and shot classification (M3, M4), movement comparison and explainable AI (M5, M6), and the AI virtual coach with adaptive training (M7, M9).

6.1 USER MANAGEMENT, HISTORY AND ADMINISTRATION

, The FastAPI application loads all models once at start-up and keeps them in memory. Each analysis endpoint runs the heavy work in a thread pool so that the event loop stays responsive. The listing below shows the video endpoint.

Passwords are hashed with bcrypt, and sign-in returns a JSON Web Token signed with a server secret and valid for 24 hours. A dependency function decodes the token on every protected request and loads the user, so each query is filtered by the user's ID (NFR-5).

Every request carries a signed token that identifies the user, and the server looks up the role on each request. Two dependencies guard the endpoints: current _p layer for analysis, training, coaching, history, dashboard and reports, and current _ admin for the administrator endpoints. They are attached to whole routers, so a new endpoint added to a player router is protected automatically. A wrong role receives 403 with a plain explanation, and an automated test checks both directions (Table 7.2). The overview for administrators lists players only and shows numbers, never another player's photos or videos.

6.2 PHOTO POSTURE ANALYSIS

A validated photo is decoded with OpenCV, converted to RGB and passed to MediaPipe Pose Landmarker in image mode. For each landmark the module keeps the normalised coordinates and the visibility score. Angles are computed only from points whose

visibility is at least 0.5; others are greyed out in the report, never guessed. The annotated image and the angle table are saved and linked to the analysis record.

, MediaPipe Pose Landmarker is created once at start-up, in image mode for photos and in video mode for clips. For three landmarks A, B and C, the angle at B is computed from the 2D vectors BA and BC.

theta = arccos((BA. BC) / (|BA| |BC|)) (6.6)

An angle is reported only when all three landmarks have visibility of at least 0.5 (FR-4). Table 6.1 lists the angles used.

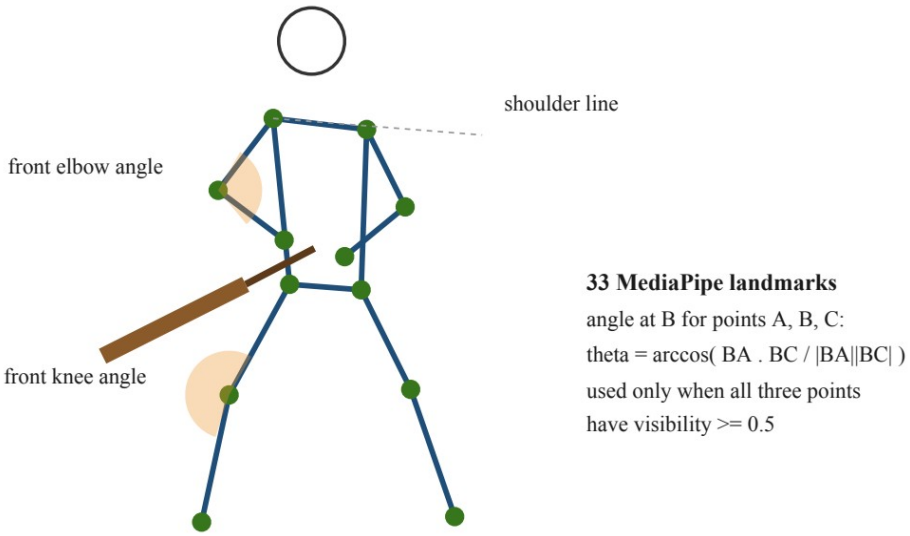


Fig: 6.1 Landmarks and joint angles used by the posture report and comparison

Table 6.1 Joint angles computed from MediaPipe landmarks

Angle	Landmarks (A - B - C)	Used in
Front elbow	shoulder - elbow - wrist	Image report, pose model, DTW
Front knee	hip - knee - ankle	Image report, pose model, DTW
Front shoulder	elbow - shoulder - hip	Pose model, DTW
Front hip	shoulder - hip - knee	Image report, pose model
Back elbow, knee, shoulder, hip	as above, other side	Pose model, DTW
Shoulder line	left shoulder - right shoulder vs horizontal	Image report
Head over front foot	nose x vs front ankle x	Image report

Every photo and clip is scored before its numbers are shown (backend/app/quality.py). Simple, explainable rules measure sharpness (variance of the Laplacian), brightness and contrast, how much of the frame the batter fills, whether the whole body is in view, the camera view (the ratio of shoulder width to torso length separates side-on, angled and front-on) and whether several people are present. The result page shows a score from 0 to 100, a chip for each check and a plain tip for each problem, such as "Turn the camera a little more to the batter's side for the clearest knee and elbow angles".

6.3 VIDEO PREPROCESSING AND SHOT CLASSIFICATION

The frame sampler chooses 15 evenly spaced indices across the clip. If a clip has fewer than 15 frames, indices repeat, as requ ired by FR-7. The same function is used for training and inference.

Upload validation checks the file extension against an allow-list, the MIME type reported by the client, the size against the configured limit, and finally whether OpenCV can open the file. Each failure maps to a specific HTTP status: 400 for a wrong type, 413 for a file that is too large and 422 for an unreadable file.

Each of the T = 15 frames x passes through the EfficientNetV2-S backbone with its classification head removed, giving a feature vector f of 1280 dimensions. The GRU [4] updates its hidden state has follows.

z = sigma(W f + U h), r = sigma(W f + U h) (6.1)

h = (1 - z) h + z tanh(W f + U (r h)) (6.2)

Additive attention [40] scores each hidden state and forms a context vector c, which is passed to the classifier. The weights a are the key-frame timeline.

e = v tanh(W h), a = exp(e) / sum exp(e), c = sum a h (6.3)

Listing: training/sc _ common.py - video model

For the pose path, each frame gives a 43-dimensional vector: eight joint angles (left and right elbow, shoulder, hip and knee) scaled to the range 0 to 1, the x and y coordinates of 17 upper- and lower-body landmarks, centred on the hip midpoint and divided by the shoulder width so that the vector does not depend on where the batter stands or how large

they appear, and one flag that is 1 when the frame is usable. A joint whose visibility is below 0.5 is set to zero instead of being guessed. A frame in which the shoulders or hips cannot be found has its flag at 0 and takes the values of the last usable frame. A GRU with 64 units summarises the sequence.

The fused model concatenates the 128-dimensional video context vector and the 64-dimensional pose summary and passes them through a two-layer perceptron (192 to 128 to 15) with dropout of 0.3. Late fusion was chosen over early fusion because it lets each stream keep its own temporal model and works directly on the cached features of both streams.

All five models are trained with the same settings so that differences come from the inputs, not the schedule (Table 6.2). The EfficientNetV2-S backbone keeps its ImageNet weights and stays frozen throughout; only the GRU, attention and classification layers are trained. This is a deliberate trade-off for the available hardware: fine-tuning the backbone on 10,000 clips of 15 frames needs far more GPU time than a free Colab session gives, while frozen features can be computed once and reused for all runs. It is also the main expected reason for any accuracy gap to the base paper, which fine-tuned the network [1].

Table 6.2 Training configuration

Setting	Value
Optimiser	AdamW, learning rate 10^{-3} with 5% linear warm-up and cosine decay, weight decay 10^{-4}
Backbone	EfficientNetV2-S with ImageNet weights, frozen (features cached once)
Batch size	64 clips for all models
Loss	Class-weighted cross-entropy, label smoothing 0.05
Epochs	Up to 40, early stopping on validation macro-F1 with patience 6
Precision	Mixed precision (float16) for feature extraction on GPU
Augmentation	Horizontal flip of training clips only, with mirrored landmarks
Random seeds	Fixed for Python, NumPy and PyTorch; three runs per model

After training, a single temperature T is learned on the validation set by minimising negative log-likelihood with the model weights frozen [9]. At inference time the calibrated probabilities are as follows; T greater than 1 softens over-confident outputs without changing the predicted class.

pi = exp(zi / T) / sum exp(z / T) (6.4)

Calibration is measured with the Expected Calibration Error over M = 15 equal-width confidence bins B m, where n is the number of test clips.

ECE = sum (|B | / n) |acc(B) - conf(B)| (6.5)

6.4 MOVEMENT COMPARISON AND EXPLAINABLE AI

Let P = (p 1,..., pn) be the player's angle vectors and R = (r 1,..., r) those of the reference clip. DTW fills a cost matrix D with the recurrence below, where d is the Euclidean distance between angle vectors over the joints visible in both frames. The total distance is D(n, m) divided by the path length, and backtracking gives the alignment used to sync the two videos side by side.

D(i, j) = d(p i, r) + min{ D(i-1, j), D(i, j-1), D(i-1, j-1) } (6.7)

Grad-CAM uses the gradient of the predicted class score with respect to the feature maps of the last EfficientNet convolutional block for each frame. The weights are global averages of the gradients; the weighted sum of feature maps, passed through ReLU and upsampled, is blended onto the frame with an OpenCV colour map. The attention weights from M4 form the key-frame timeline. The interface shows heatmap, pose skeleton and timeline side by side.

The real heatmaps produced by the trained model are shown on the video result page in Appendix 2.

For class c and the feature maps A of the last convolutional block, Grad-CAM [8] computes channel weights by averaging gradients over the spatial positions and forms the heatmap as follows, where Z is the number of spatial positions.

w c = (1/Z) sum dyc / dA ij, Lc = ReLU(sum w c Ak) (6.8)

6.5 AI VIRTUAL COACH AND ADAPTIVE TRAINING

The coach builds its prompt from three parts: fixed system rules, the stored analysis as compact JSON, and the last six messages. Images are never sent. After the model replies, a guard extracts every number in the reply and checks that it appears in the stored result; if not, the reply is regenerated once with a stricter instruction, and if it fails again a safe fallback is returned.

Listing: backend/app/coach.py - system rules and guard

Each morning the plan is built from the player's last five measured photographs and clips. For each joint the engine averages only the angles that passed the visibility rule. A photograph is one moment, so the more bent side is used for knees, hips and elbows, which in a side-on stroke is usually the front side. A clip covers the whole stroke, so its deepest knee and hip bend and its highest arm position are used, while the elbow is taken at the frame of most movement, because elbows fold right up at the top of every backlift and their minimum would trigger the elbow drill for every clip. Table 6.3 lists the rules. The thresholds are coaching rules of thumb applied to 2D estimates; they decide what to practise, never whether a technique is right, and the page says so under the plan. Up to three weak-spot drills are taken in the order of the table, and the plan is filled to three with general drills (shadow batting, footwork, finish balance, drop-ba ll defence and ball-watching), rotated by date so that the same filler does not appear every day.

Every drill carries the reason it was chosen, written with the measured value, for example "Your more-bent knee averaged 168 degrees across your last 4 photos". This keeps the plan as explainable as the analyses it comes from.

Table 6.3 Weak-spot rules used to choose drills

Measurement over last five photos and clips	Condition	Drill added
More-bent knee, average	above 160 degrees	Front-knee bend shadow drill
Shoulder-line tilt, average	above 20 degrees	Level-shoulders mirror drill
More-bent hip, average	above 165 degrees	Hip-hinge stance hold
More-bent elbow, average	below 80 degrees	Front-elbow lead drill
Shoulder angle, average of both sides	below 35 degrees	Backlift hold
Body points visible (photos), average	below 26 of 33	Camera set-up photo
Recent clip confidence	below 60%	Shadow practice of that shot

A practice day is any day with an analysis, a finished drill or a check-in. The current streak counts consecutive practice days up to today, and survives until the end of today so that a player is not penalised in the morning for not having practised yet. Experience points are earned for each analysis (10), finished drill (15), check-in (5), quiz answer (10 if right, 3 for trying), saved live session (5) and practice day (5, plus 5 on a streak), and levels follow the requirements in Table 6.4. A level needs all lower levels first, and because every requirement is a cumulative count, a level once reached is never lost. All of these values are recomputed from the stored rows on each request instead of being stored as counters.

Table 6.4 Levels and their requirements

Level	Name	Requirement (in addition to all lower levels)
1	Net Rookie	Create an account
2	Club Batter	5 analyses and a best streak of 3 days
3	District	20 analyses and 5 photos with all 8 joints measured
4	State	40 analyses and 2 completed plan weeks (all drills done on at least 5 days of a week)
5	Pro	75 analyses and a best streak of 21 days

The live mirror page (frontend/live.html) runs the MediaPipe pose model in the browser through WebAssembly, so camera frames are analysed in memory and never uploaded. Repetitions and holds are counted with simple hysteresis rules on the drill's target angle

(enter the target zone, stay there for a set time, leave it again), the voice coach uses the browser's speech synthesis, and voice commands use its speech recognition. The ghost of a reference clip is scaled to the player's torso and anchored at the hips, and the live match compares the player's eight angles with the ghost's at the same moment, allowing two frames of timing error either way. Phone mode (run _p hone.py) serves the application over HTTPS on the local Wi-Fi with a certificate made for the computer's address, because phones allow the camera only on secure pages. The features are shown in Appendix 2.

CHAPTER 7

TESTING

Testing checks that the system meets the requirements of Chapter 4 and finds defects before users do. This chapter describes the testing strategy, the levels of testing, and the test cases for each module. Test case IDs trace back to the functional requirements (FR) and non-functional requirements (NFR).

7.1 TESTING STRATEGY

Testing is carried out at four levels (Fig. 7.1). Unit tests are written with pytest for the back end and run on every commit. Integration and system tests use FastAPI's TestClient and a separate PostgreSQL test database. Acceptance testing is done with players and a coach using the deployed application. Model testing, which measures accuracy and calibration rather than pass or fail, is reported separately in Section 7.6.

- Black-box testing checks inputs and outputs against the requirements without looking at the code: valid and invalid uploads, boundary file sizes, wrong tokens.
- White-box testing exercises internal paths: the short-clip branch of the frame sampler, the low-visibility branch of the angle function, the regenerate branch of the coach guard.
- Regression testing reruns the full suite after each change to make sure that fixed defects stay fixed.
- Performance testing measures response time per clip on GPU and CPU (NFR-1).

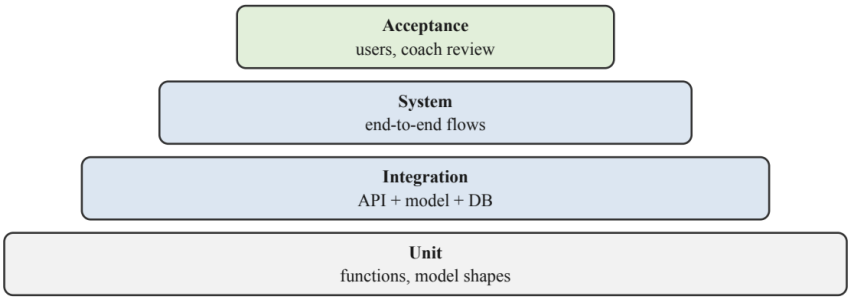


Fig: 7.1 Levels of testing used in the project

7.2 UNIT TESTING

Unit tests check single functions and model components in isolation, with fixed inputs and known expected outputs. They run automatically with pytest. Table 7.1 lists the unit test cases.

Table 7.1 Unit test cases

ID	Unit	Input	Expected result	Result
UT-01	sample_frames	30-frame clip	Array of shape (15, 224, 224, 3)	Pass
UT-02	sample_frames	8-frame clip	15 frames, indices repeated	Pass
UT-03	sample_frames	Empty file	ValueError raised	Pass
UT-04	to_tensor	(15, 224, 224, 3) uint8	Tensor (15, 3, 224, 224), mean near 0	Pass
UT-05	joint_angle	Points at right angle	90.0 degrees (within 0.1)	Pass
UT-06	joint_angle	One point visibility 0.3	None	Pass
UT-07	dtw	Identical sequences	Distance 0, diagonal path	Pass
UT-08	dtw	Sequence and its 2x time-stretched copy	Distance near 0	Pass
UT-09	ShotNet.forward	Random (2, 15, 3, 224, 224)	Logits (2, 15), attention rows sum to 1	Pass
UT-10	TemperatureScaler	Over-confident logits	T greater than 1, NLL reduced	Pass
UT-11	grad_cam	One clip, class 0	15 maps of 7 x 7 in [0, 1]	Pass
UT-12	numbers_grounded	Reply with number not in result	False	Pass
UT-13	hash_password / verify	Correct and wrong password	True / False	Pass
UT-14	streaks	Practice days with a gap	Current and best streak counted correctly	Pass
UT-15	level_info	Counts for each level	Correct level; level 3 not reached without level 2	Pass
UT-16	dose	Same drill at three difficulty levels	2 x 7, 3 x 10, 4 x 13 repetitions	Pass
UT-17	template_summary	One week of statistics	Sentence uses only the given numbers	Pass
UT-18	quiz bank	All 22 questions	Unique ids, four options, valid answer index	Pass

7.3 INTEGRATION TESTING

Integration tests send real HTTP requests to the FastAPI application with a test database and check that the API, the analysis services and the database work together. Table 7.2 lists the integration test cases.

Table 7.2 Integration test cases

ID	Scenario	Expected result	Result
IT-01	Register then sign in with same credentials	201 Created, then 200 with token	Pass
IT-02	Call /api/history without token	401 Unauthorized	Pass
IT-03	Upload valid JPG to /api/image/analyse	200, landmarks count, angles, annotated image path	Pass
IT-04	Upload valid MP4 to /api/video/analyse	200, shot, confidence, top-3, heatmap path, latency	Pass
IT-05	Video analysis writes media and analysis rows	Both rows present, foreign key correct	Pass
IT-06	Compare analysis with reference clip	200, distance and per-joint differences	Pass
IT-07	Coach question on stored analysis	200, reply stored in coach_messages	Pass
IT-08	Analysis raises error mid-way	500, no partial rows, temp file deleted	Not run
IT-09	User A requests user B's analysis	404 Not Found	Pass
IT-10	Check-in with soreness that needs rest, then mild soreness	Rest-day plan, then three easy drills	Pass
IT-11	Tick a drill, then check in again	XP and streak updated; finished plan kept	Pass
IT-12	Answer the question of the day twice	First 200 with XP, second 409	Pass
IT-13	User B ticks user A's drill	404 Not Found	Pass
IT-14	Administrator calls player endpoints; player calls administrator endpoints	403 in both directions; overview lists players only	Pass
IT-15	Upload a clip with no shot model installed	200 movement analysis: 15 frames, skeleton frames, PDF report	Pass

7.4 SYSTEM TESTING

System tests follow complete user journeys through the browser against the deployed stack.

Table 7.3 System test cases (FR and NFR refer to the functional and non-functional requirements of the project specification)

ID	Requirement	Steps	Expected result	Result
ST-01	FR-3	Upload a .gif in image mode	Message "Only JPG or PNG images are allowed"	Pass
ST-02	FR-6	Upload a 70 MB video	413 with "File is larger than 50 MB"	Pass
ST-03	FR-6	Upload a renamed text file as .mp4	422 "Video could not be read"	Pass
ST-04	FR-4, FR-5	Upload side-on batting photo	Skeleton drawn, angles shown, hidden points grey	Pass
ST-05	FR-8, FR-10	Upload clip, choose each model	Shot, confidence and top-3 for each model	Pass
ST-06	FR-9	Upload a non-cricket clip	Result marked "uncertain"	Not run
ST-07	FR-11, FR-12	Open result page	Heatmaps for 15 frames, timeline chart	Pass
ST-08	FR-13	Pick reference clip of same shot	Synced side-by-side view, angle-difference chart	Pass
ST-09	FR-14	Ask "Why not a hook?"	Reply with Measured and General tip parts	Pass
ST-10	FR-14, NFR-10	Ask about an uncertain clip	Coach says there is not enough data	Not run
ST-11	FR-15, FR-16	Analyse three clips, open dashboard	Counts, shot mix and trend updated	Pass
ST-12	FR-17	Download CSV and HTML report	Files open with correct rows and values	Pass
ST-13	NFR-6	Open all pages at 360 px width	No horizontal scroll, all controls usable	Pass
ST-14	FR-19, FR-21	Open Training, tick all three drills	Drills turn green, XP rises, ring shows 3/3, calendar day marked	Pass
ST-15	FR-20	Check in with "Sore, I need rest"	Rest day shown with advice to see a professional	Pass
ST-16	FR-23	Open Progress after a week of use	Weekly numbers and a summary that uses only those numbers	Pass
ST-17	FR-24	Upload a broadcast clip with camera zoom	Skeleton on frames, peak movement inside the stroke, not at the zoom	Pass
ST-18	FR-25	Sign in as administrator, open the Training page	Redirected to the administrator overview	Pass

7.5 ACCEPTANCE TESTING

Acceptance testing is planned with a small group of club players and one coach before the system is offered to a club; within this project the same tasks were carried out by the

project team on the finished build, and all of them were completed. Each participant completes five tasks without help: register, analyse a photo, analyse a clip, compare with a reference and ask the coach two questions. For each task we record whether it was completed, how long it took and any confusion observed. Participants then rate clarity of results, usefulness of the heatmap and timeline, and usefulness of the coach on a five-point scale. Coach replies are also reviewed by the project team against the stored results, counting any statement that is not supported by the data (experiment E8).

7.6 PERFORMANCE EVALUATION OF THE SHOT CLASSIFIERS

Let TP c, FP and FN be the true positives, false positives and false negatives for class c, and C = 15 the number of classes. The following metrics are used.

Accuracy = (number of correctly classified clips) / (total clips) (8.1) Precision = TP / (TP + FP), Recall = TP / (TP + FN) (8.2) F1 = 2 Precision Recall / (Precision + Recall) (8.3)

Macro-F1 = (1/C) sum F1 (8.4)

Macro-F1 gives each shot equal weight, which matters because the largest class in CricShot10k is 4.26 times the smallest [1]. Top-k accuracy counts a clip as correct if the true class is among the k highest-scoring classes. Calibration is measured with ECE (Equation 6.5). Latency is the mean time per clip in milliseconds.

7.6.1 Classification results

, Main finding. Adding body pose to video improved every metric. The fused model E3 reached 63.93% top-1 accuracy and 62.24% macro-F1, against 56.17% and 55.33% for video alone: a gain of 7.8 points in accuracy and 6.9 points in macro-F1. Pose alone (E2) was weaker than video alone, as expected and as found in [1] and [6], because a skeleton carries no information about the bat, the ball or the field. The two streams therefore carry complementary information: the video stream sees bat and ball, the pose stream sees how the body moves, and together they do better than either. The correct shot was among the top three predictions for about nine clips in ten with either fused model.

Is the gain real? Because all models are tested on the same clips, McNemar's exact test was used to compare them (Table 7.5). It counts the clips that one model gets right and

the other gets wrong. Of the clips where E1 and E3 disagreed, E3 was right on 195 and E1 on 95; the chance of such a difference arising by luck is below one in a hundred million ($p = 4.3 \times 10^{-9}$). The improvement from pose is therefore statistically significant, not an accident of the test set.

2D against 3D pose (E9). Replacing the 2D pose features with MediaPipe's 3D world landmarks did not improve classification: E9a and E2 are within half a point of each other, E9b is slightly below E3, and neither difference is significant. A likely reason is that the 3D landmarks are themselves estimated from a single 2D image, so they cannot add information that the 2D landmarks do not already contain; they mainly reorganise it. The 3D features are still valuable for measurement, as Section 7.6.4 shows.

Comparison with the base paper. The video-only model is 33 points below the 89% reported for CricShotNet [1]. Three differences explain most of this gap: the backbone was kept frozen with ImageNet weights to fit free GPU time, whereas [1] fine-tuned it on cricket frames; the batter was not cropped or segmented with YOLO, which [1] reports as worth several points; and the split is different, because the authors' lists are not published. The gap is a cost of the training budget, not of the pipeline, and fine-tuning is the first item of future work. For the research question, which asks whether pose helps, all five models share the same handicap, so the comparison between them is fair.

Table 7.4 Classification results on the CricShot10k test split (1,289 clips)

Model	Top-1 (%)	Top-3 (%)	Macro-F1 (%)	Precision (%)	Recall (%)
CricShotNet, reported in [1]	89	99	-	-	-
E1 Video only	56.17	86.81	55.33	56.54	56.10
E2 Pose only (2D)	49.57	76.65	47.00	47.84	49.46
E3 Video + pose (2D)	**63.93**	89.60	**62.24**	**62.30**	**62.72**
E9a Pose only (3D)	49.19	77.66	46.68	47.52	49.59
E9b Video + pose (3D)	62.53	**90.15**	60.69	60.82	61.81

Table 7.5 McNemar exact test between models on the same test clips

Comparison	Only first right	Only second right	p-value	Significant ($p < 0.05$)
E3 Video + pose (2D) vs E1 Video only	95	195	4.3×10^{-9}	Yes
E9b Video + pose (3D) vs E1 Video only	109	191	2.5×10^{-6}	Yes
E9b Video + pose (3D) vs E3 Video + pose (2D)	122	104	0.26	No
E9a Pose only (3D) vs E2 Pose only (2D)	129	124	0.80	No

7.6.2 Per-class analysis

The easiest shots were the flick (F1 0.79), the defensive block (0.77) and the sweep (0.75), which have distinctive body positions: an upright, still body for the block and a low, kneeling body for the sweep. The hardest was the late cut (0.44)

, a short and subtle movement with few training clips (220). The lofted shots were often confused with their ground-played counterparts, because the body action is similar and the difference lies mainly in the bat angle and the flight of the ball, which a frozen backbone at 224 x 224 pixels sees poorly. This agrees with the base paper's observation that shots sharing a body action, such as the pull and the hook, are the hardest to separate [1].

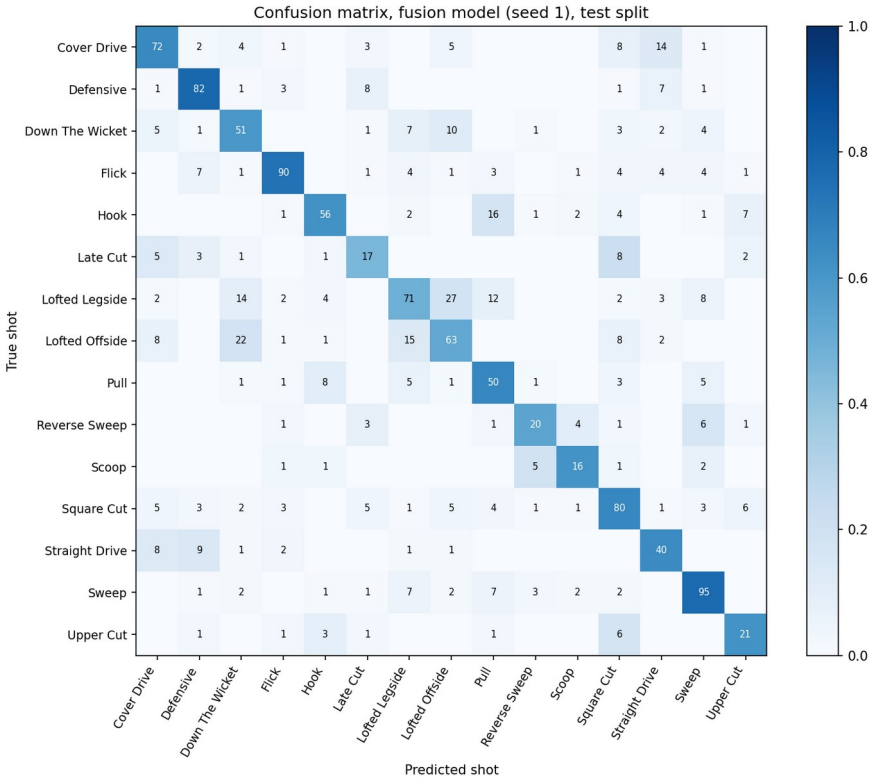


Fig: 7.2 Confusion matrix of the best model on the test split; rows are true shots, columns predicted shots

7.6.3 Calibration

Temperature scaling lowered the expected calibration error of every model, by between 22% (E3) and 44% (E2), without changing any prediction. The video and fused models were slightly over-confident (T greater than 1, so scores are so ftened), while the pose-only models were under-confident (T less than 1). After scaling, a confidence of 80% shown in the app corresponds closely to an 80% chance of being right, which is what makes the uncertainty flag meaningful.

Table 7.6 Calibration before and after temperature scaling

Model	Temperature T	ECE before	ECE after
E1 Video only	1.076	0.0421	0.0286
E2 Pose only (2D)	0.899	0.0833	0.0468
E3 Video + pose (2D)	1.092	0.0400	0.0311
E9a Pose only (3D)	0.894	0.0641	0.0437
E9b Video + pose (3D)	1.106	0.0539	0.0397

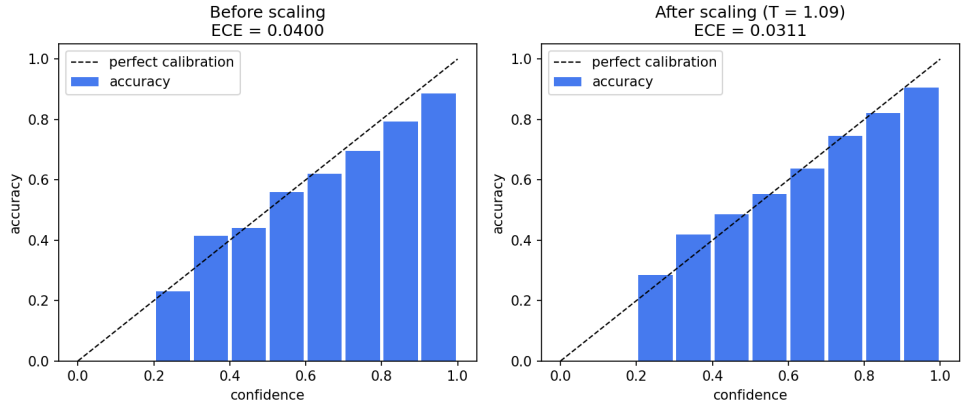


Fig: 7.3 Reliability diagram of the best model before and after temperature scaling

7.6.4 Robustness across camera views and sharpness

Each test clip was labelled by camera view from the ratio of shoulder width to torso length (side-on, angled or front-on, the same rule as the application's quality check) and by image sharpness. Table 7.7 shows the macro-F1 of each model in each group.

The gain from adding pose was largest for front-on clips, where macro-F1 rose from 47.3% to 57.6%, which suggests that the body position supplies information that the picture of a batter facing the camera hides. Side-on broadcast clips were the hardest group for all models; they are also the smallest group (159 clips), so their scores are less certain. Blurry frames did not hurt the fused model: pose landmarks are found from the overall body shape, which survives moderate blur better than fine texture does.

The same clips were used to measure how far 2D joint angles, read from the picture, differ from 3D angles computed from MediaPipe's world landmarks. The mean gap was 16.2 degrees for side-on clips, 20.0 for angled clips and 24.8 for front-on clips. A 2D angle is therefore most trustworthy when the camera is side-on, as the application advises, and the 3D view in the application is most useful exactly when the camera is not. This is why the result pages show both the 2D and the 3D value of each angle.

Table 7.7 Macro-F1 (%) by camera view and image sharpness

Group	Clips	E1 Video	E2 Pose 2D	E3 Video + pose 2D	E9a Pose 3D	E9b Video + pose 3D
Angled view	797	53.1	47.5	60.7	47.0	60.1
Front-on view	332	47.3	34.0	57.6	30.6	48.7
Side-on view	159	42.4	34.9	44.0	36.2	41.7
Sharp frames	996	55.1	45.7	61.2	45.3	59.7
Soft frames (sharpness < 90)	293	54.8	50.6	64.9	50.3	63.0

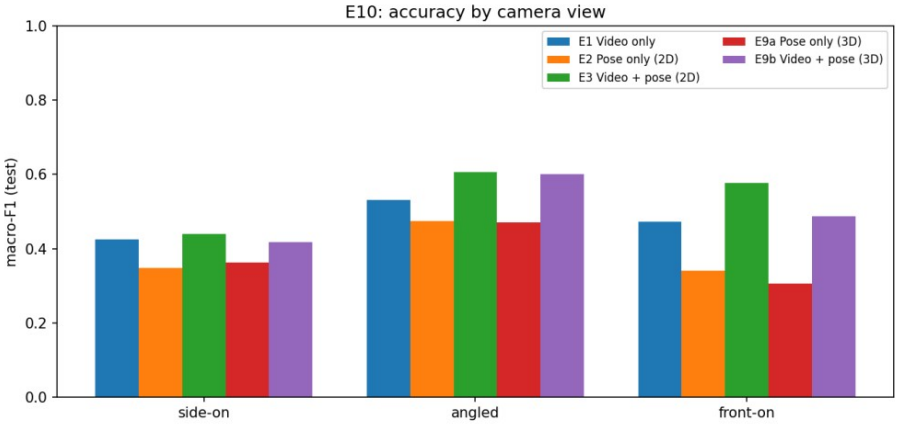


Fig. 8.2 Macro-F1 of the five models by camera view

Produced by the training notebook.

7.7 TEST SUMMARY

The automated suite of 64 tests (pytest) passed in full on the final build, and the system tests were also carried out by hand in Chrome on a laptop and at phone width. Three cases are marked Not run: IT-08 (an internal failure part-way through an analysis, which is handled in code but was not forced in a test), ST-06 and ST-10 (a non-cricket clip and the coach's answer to it), which need clips outside the dataset and are left for acceptance testing. Several defects found during testing were fixed and retested, among them the bowler being tracked instead of the batter in broadcast clips, camera zooms being counted as body movement, and a broken camera start in one browser. Model quality is not a pass/fail matter, so it is evaluated through experiments E1 to E10 in the next chapter.

CHAPTER 8

ADVANTAGES OF THE PROPOSED SYSTEM

Compared with manual shot tagging and with the existing CricShotNet system, SmartCricket AI offers the following advantages.

- **Photo and video analysis.** A quick posture report from a single photograph, and a full shot analysis from a short clip, in the same application.
- **Proven benefit of body pose.** Fusing body pose with the video stream raised top-1 accuracy from 56.2% to 63.9% on the same test clips, a statistically significant gain (McNemar test, $p < 10^{-8}$).
- **Answers that can be checked.** Grad-CAM heatmaps show where the model looked, the attention timeline shows which frame mattered most, and the top-3 list shows the close alternatives.
- **Honest confidence.** Temperature scaling makes a stated confidence match the real chance of being right, and uncertain clips are flagged instead of being presented as facts.
- **Measured posture feedback.** Eight joint angles are measured only when the joints are clearly visible, so the report never guesses at hidden joints.
- **Comparison with a model player.** Dynamic Time Warping lines up the player's swing with a reference clip of the same shot, even when the two were played at different speeds.
- **Grounded virtual coach.** The language model sees only the stored numbers, separates measured facts from general tips, and every number in its reply is checked.
- **Daily, adaptive training.** Three drills a day are chosen from the player's own weak joints, adapt to soreness and completion, and are rewarded with experience points, streaks and levels.

- **Live practice without uploads.** The live mirror runs pose estimation in the browser, counts repetitions and compares the player with a model player's skeleton, without sending any video to the server.
- **Input quality check.** Every upload is checked for sharpness, light, batter size, camera view and other people, with a tip for each problem.
- **Low cost.** The application runs on an ordinary laptop CPU, uses only free and open-source software, and the models are trained on a free cloud GPU.
- **Complete platform.** Player and administrator roles, history, dashboards, CSV and PDF reports and an administrator console make the work usable outside a research setting.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 CONCLUSION

This project set out to analyse cricket batting automatically from either a single photograph or a short video clip, and to make the analysis understandable and useful to players and coaches. SmartCricket AI builds on the CricShot10k dataset and the CricShotNet model [1], which provide 10,086 labelled clips in 15 shot classes and a strong EfficientNetV2-S + GRU baseline with 89% top-1 accuracy.

Around that foundation the project designed and implemented a complete system. Image mode turns a photograph into a posture report of joint angles from MediaPipe landmarks, report ing only points that are clearly visible. Video mode samples 15 frames, c lassifies the shot with a video-only, pose-on ly or fused model, calibrates the confidence with temperature scaling and flags uncertain clips. Each prediction is explained with Grad-CAM heatmaps and an attention-based key-frame timeline, and the player's movement is compared with a reference clip using Dynamic Time Warping. An AI virtual coach, grounded only in the stored results, explains each outcome in plain language and keeps measured facts apart from general advice. An adaptive training coach turns the same measurements into a daily plan of drills, adjusts it to the player's consistency and soreness, and keeps players engaged with levels, streaks, a daily quiz and a weekly report. The platform is delivered as a responsive web application with a FastAPI back end, PostgreSQL storage, session history, a dashboard and downloadable reports.

The evaluation answered the main research question. On 1,289 test clips, fusing video with body pose raised top-1 accuracy from 56.2% to 63.9% and macro-F1 from 55.3% to 62.2%, a gain that McNemar's test showed to be statistically significant and that was largest for front-on camera views. 3D pose features did not improve recognition over 2D, but they revealed that a single camera distorts joint angles by 16 to 25 degrees depending on the view, which the application now shows to the player. Temperature scaling made the confidence of all five models more honest. The absolute accuracy is below the base paper's 89% because the backbone was kept frozen and the batter was not cropped,

choices made to fit a free GPU budget; these affect all five models equally and do not change the comparison between them.

9.2 LIMITATIONS

- Only the 15 shot classes of CricShot10k are recognised; other strokes are forced into the nearest class or flagged as uncertain.
- The training data is broadcast footage; phone and academy video filmed from other angles may reduce accuracy.
- Joint angles are 2D estimates from a single camera and depend on camera angle; they are not biomechanical measurements.
- The coach depends on an external language model service and needs network access.
- The system analyses one batter per clip and does not track the ball.

9.3 FUTURE WORK

1. Video transformers. Evaluate TimeSformer [30] and Video Swin Transformer [31], as suggested in [1], and compare them with the GRU-based models on accuracy and latency.
2. More shot classes. Extend the label set to the leave, leg glance and back-foot punch, using the same automatic clip-cutting method as [1].
3. Domain adaptation for phone video. Collect a small labelled set of net-session clips and fine-tune, or use unsupervised domain adaptation, to close the gap between broadcast and amateur footage.
4. Skeleton graph networks. Replace the pose GRU with an ST-GCN [34] that models the body as a graph of joints.
5. 3D pose and multi-view. Use MediaPipe's world coordinates or two synchronised cameras to obtain view-independent joint angles.
6. Fine-tune the backbone and crop the batter. Unfreeze EfficientNetV2-S on cricket frames and add YOLO batter cropping, which together are expected to

close most of the gap to the base paper's 89%, and run the live shot recognition with a smaller model directly on the phone.

7. Ball tracking. Add ball detection and trajectory estimation to link the shot to where the ball went, enabling wagon wheels and scoring-zone charts.
8. Multilingual coach. Offer coaching replies in regional languages such as Tamil and Hindi.

9.4 WORK SCHEDULE FOR PHASE II

Table 9.1 gives the plan for Phase II of the project.

Table 9.1 Work schedule for Phase II

Period	Work planned
Month 1	Train the improved classifiers: RegNetY-16GF backbone with a pose-based batter crop and three seeds; then fine-tune the last backbone blocks on cricket frames
Month 2	Collect and label clips for new shots (slog sweep, paddle sweep, switch hit, leg glance, leave) and a real-world test set of phone and net-session videos
Month 3	Evaluate on CricShot10k and on the real-world test set; add a single-frame shot model for photos with its own measured accuracy
Month 4	User study with players and coaches, performance tuning, final testing and thesis writing

APPENDIX 1

SAMPLE CODINGS

This appendix gives selected parts of the source code. The full code is organised as a Python package (backend), a shared training module (training/sc_common.py) used by both the Colab notebook and the web application, and the HTML, CSS and JavaScript front end.

A1.1 Shot classifier: attention, video stream and late fusion (training/sc_common.py)

```
class Attention(nn.Module):
    """Additive attention over GRU outputs; the weights are the key-frame timeline."""

    def __init__(self, hidden: int):
        super().__init__()
        self.score = nn.Sequential(nn.Linear(hidden, 64), nn.Tanh(), nn.Linear(64, 1))

    def forward(self, h):                       # h: (B, T, H)
        a = torch.softmax(self.score(h).squeeze(-1), dim=1)
        return (a.unsqueeze(-1) * h).sum(1), a

class VideoHead(nn.Module):
    """E1: per-frame EfficientNetV2-S features -> GRU(128) -> attention -> classes."""

    def __init__(self, n_classes=len(BASE_CLASSES), hidden=128, in_dim=FEAT_DIM,
dropout=0.3):
        super().__init__()
        self.drop_in = nn.Dropout(0.2)
        self.gru = nn.GRU(in_dim, hidden, batch_first=True)
        self.att = Attention(hidden)
        self.out_dim = hidden
        self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(hidden, n_classes))

    def encode(self, x):
        h, _ = self.gru(self.drop_in(x))
        return self.att(h)

    def forward(self, x, pose=None):
        c, a = self.encode(x)
        return self.head(c), a

class FusionNet(nn.Module):
    """E3: late fusion of the video and pose summaries -> MLP (192 -> 128 -> classes)."""

    def __init__(self, n_classes=len(BASE_CLASSES), pose_dim=POSE_DIM,
feat_dim=FEAT_DIM):
        super().__init__()
        self.video = VideoHead(n_classes, in_dim=feat_dim)
        self.pose = PoseNet(n_classes, in_dim=pose_dim)
        self.head = nn.Sequential(nn.Linear(self.video.out_dim + self.pose.out_dim, 128),
nn.ReLU(),
                                  nn.Dropout(0.3), nn.Linear(128, n_classes))

    def forward(self, x, pose):
        cv, av = self.video.encode(x)
        cp, _ = self.pose.encode(pose)
        return self.head(torch.cat([cv, cp], 1)), av
```

A1.2 Batter crop from the pose landmarks (training/sc_common.py)

```
def batter_box(lm: np.ndarray, w: int, h: int, margin: float = 0.3, min_frames: int = 3):
    """One square box around the batter for the whole clip, from the pose landmarks (n,
33, 3) in pixels.


    Uses frames where at least 8 points are clearly visible; the 10th/90th percentiles of
the body edges ignore a
    stray detection, and the margin leaves room for the bat. None (use the full frame)
when the body is not found
    in enough frames. The box is fixed, so the movement between frames is kept."""
    boxes = []
    for f in lm:
        v = f[:, 2] >= MIN_VIS
        if v.sum() >= 8:
            boxes.append((f[v, 0].min(), f[v, 1].min(), f[v, 0].max(), f[v, 1].max()))
    if len(boxes) < min_frames:
        return None
    b = np.array(boxes)
    x1, y1 = np.percentile(b[:, 0], 10), np.percentile(b[:, 1], 10)
    x2, y2 = np.percentile(b[:, 2], 90), np.percentile(b[:, 3], 90)
    side = max(x2 - x1, y2 - y1) * (1 + 2 * margin)
    side = min(max(side, 0.25 * min(w, h)), min(w, h))   # not tiny, and never larger
than the frame
    cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
    x0 = int(np.clip(cx - side / 2, 0, w - side)); y0 = int(np.clip(cy - side / 2, 0, h -
side))
    return x0, y0, x0 + int(side), y0 + int(side)


def crop_frames(frames: list[np.ndarray], box) -> list[np.ndarray]:
    if box is None:
        return frames
    x1, y1, x2, y2 = box
    return [f[y1:y2, x1:x2] for f in frames]
```

A1.3 Streaks of the training coach (backend/app/trainer.py)

```
def active_days(c: Ctx) -> Counter:
    """A practice day is any day with an analysis, a finished drill or a check-in."""
    days = Counter(local_day(a.created_at) for a in c.analyses)
    days.update(d.day for d in c.drills if d.done)
    days.update(c.logs.keys())
    days.update(s.day for s in c.live)
    return days


def streaks(days, day: date) -> tuple[int, int]:
    cur, d = 0, day if day in days else day - timedelta(days=1)  # a streak survives
until the end of today
    while d in days:
        cur, d = cur + 1, d - timedelta(days=1)
    best = run = 0
    prev = None
    for d in sorted(days):
        run = run + 1 if prev and d - prev == timedelta(days=1) else 1
        best, prev = max(best, run), d
    return cur, best
```

APPENDIX 2

SNAP SHOTS

The screenshots below were taken from the working SmartCricket AI application.

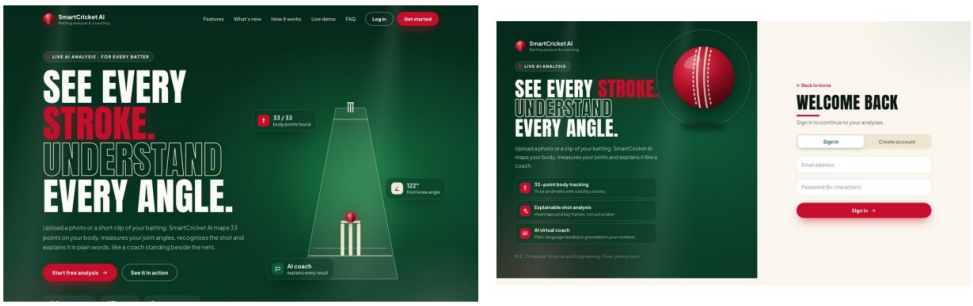


Fig: A2.1 Public landing page (left) and sign-in page (right)

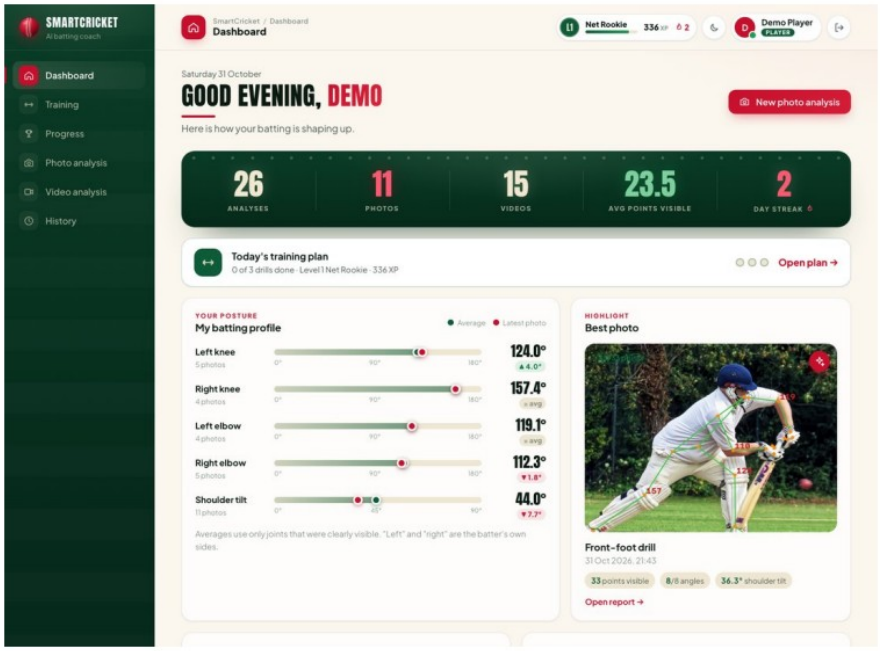


Fig: A2.2 Player dashboard with scoreboard, today's plan, batting profile and best photo

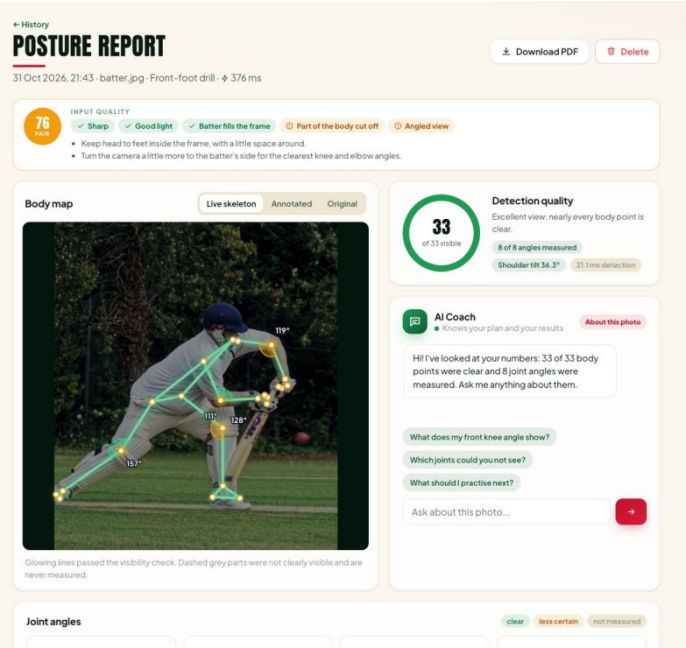


Fig: A2.3 Photo posture report: quality score, body map with measured angles and detection quality

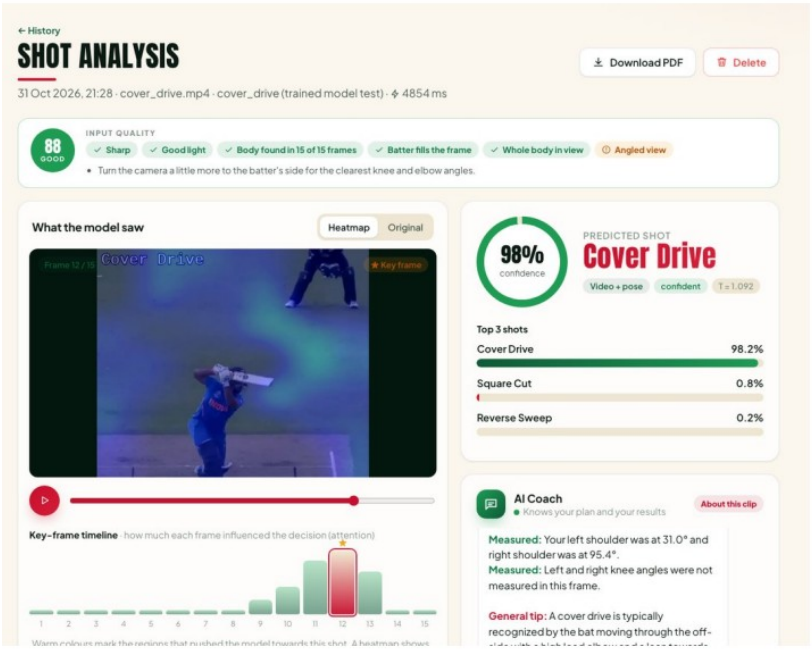


Fig: A2.4 Video shot analysis: heatmap, attention timeline and predicted shot with calibrated confidence

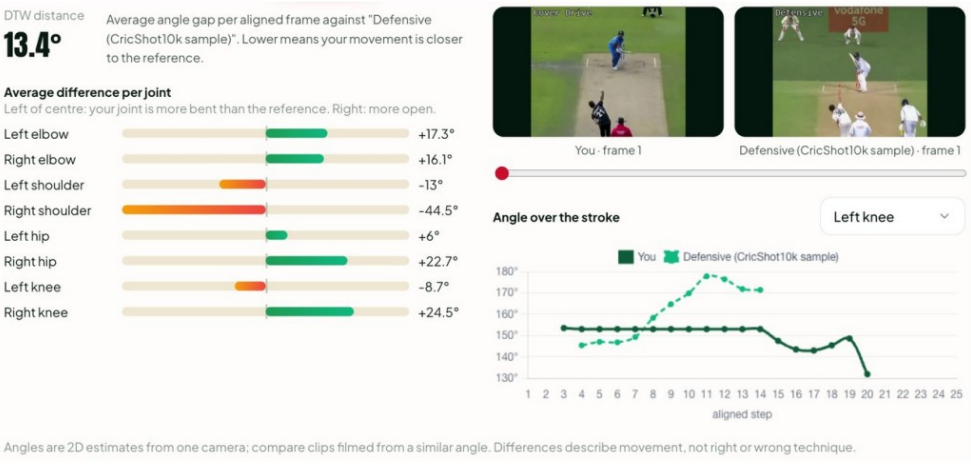


Fig: A2.5 Comparison with a reference clip: DTW distance and per-joint differences

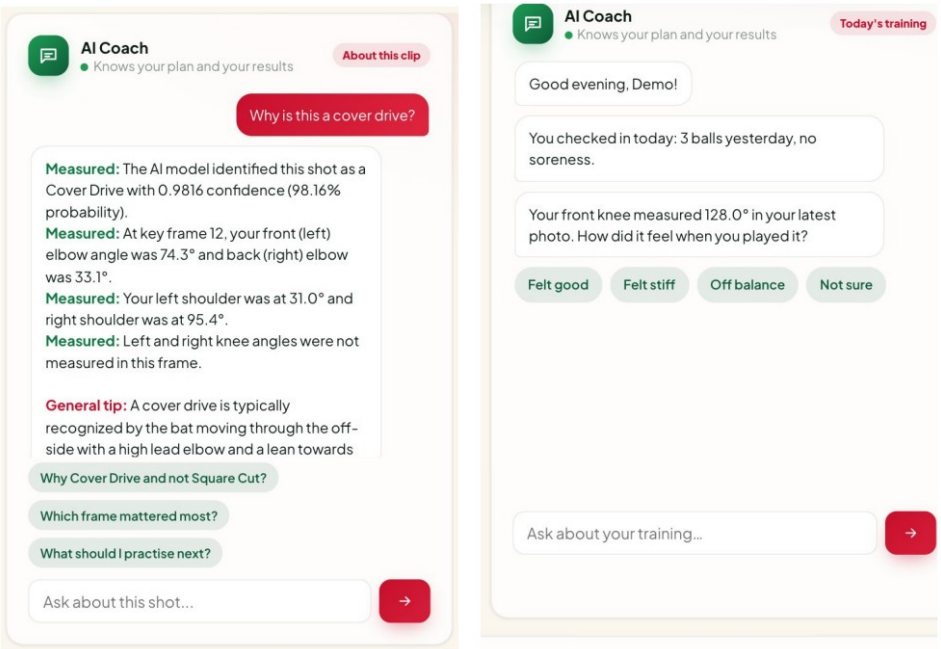


Fig: A2.6 AI Coach explaining a cover drive with measured facts and general tips

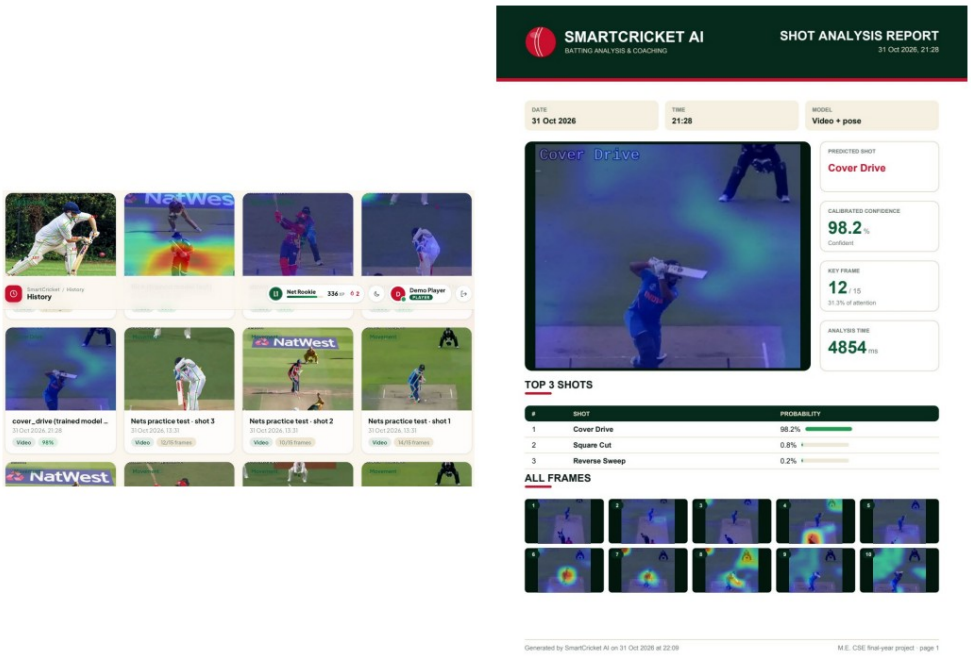


Fig: A2.7 History of analyses (left) and the one-click PDF report of a shot analysis (right)

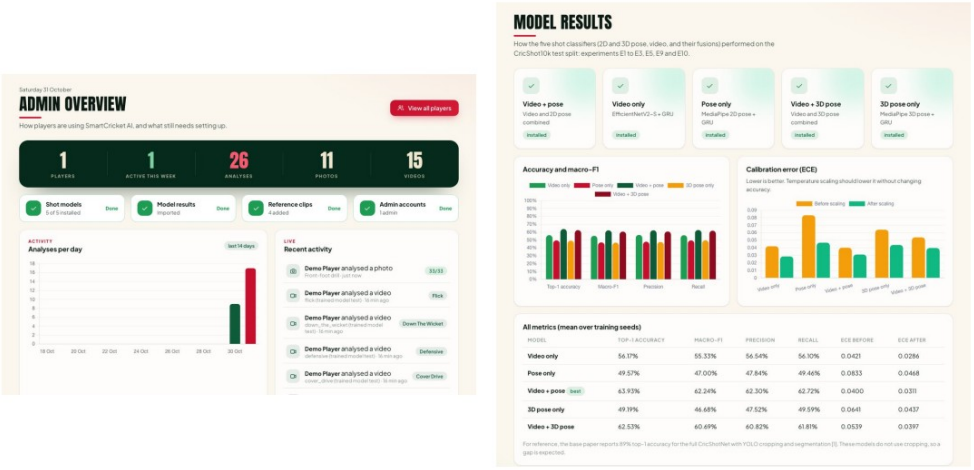


Fig: A2.8 Administrator overview (left) and model results (right)

REFERENCES

[1] M. K. Dihan, Abdullah, Amina, and S. Ahmed, "CricShot10k: A large-scale video dataset for cricket shot classification," IEEE Access, vol. 14, pp. 23428-23447, 2026, doi: 10.1109/ACCESS.2026.3663220.

[2] M. K. Dihan et al., "CricShot10k dataset and code," GitHub repository. [Online]. Available: https://github.com/Dihan69/CricShot10k

[3] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in Proc. Int. Conf. Mach. Learn. (ICML), 2021, pp. 10096-10106.

[4] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. EMNLP, 2014, pp. 1724-1734.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735-1780, 1997.

[6] C. G. Rao, N. S. H. Varma, P. Mazumder, and P. Ramasamy, "Cricket shot classification from video using body pose landmarks," in Proc. 5th Int. Conf. Adv. Electr., Comput., Commun. Sustain. Technol. (ICAECT), Jan. 2025, pp. 1-8.

[7] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), Oct. 2019, pp. 1314-1324.

[8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618-626.

[9] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 1321-1330.

[10] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," IEEE Trans. Acoust., Speech, Signal Process., vol. 26, no. 1, pp. 43-49, Feb. 1978.

[11] S. W. Hoque, M. A. Habib, A. Shome, and S. Sakib, "KUCricShot: A dataset for developing detection model of cricket shots," in Proc. 26th Int. Conf. Comput. Inf. Technol. (ICCIT), Dec. 2023, pp. 1-6.

[12] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba, "CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit," Sensors, vol. 21, no. 8, p. 2846, Apr. 2021.

[13] M. F. A. Foysal, M. S. Islam, A. Karim, and N. Neehal, "Shot-Net: A convolutional neural network for classifying different cricket shots," in Proc. 2nd Int. Conf. Recent Trends Image Process. Pattern Recognit., Springer, Dec. 2019, pp. 111-120.

[14] J. B. Fernandes, P. S. Ram, P. M. V. Yadav, and K. P. Kumar, "Cricket shot detection using 2D CNN," in Proc. 7th Int. Conf. Intell. Comput. Control Syst. (ICICCS), May 2023, pp. 608-612.

[15] A. Suryavanshi, V. Kukreja, S. Mehta, A. Garg, and P. Tiwari, "Innovative innings: Exploring cricket shot categories with CNN-SVM methodologies," in Proc. IEEE Int. Conf. Inf. Technol., Electron. Intell. Commun. Syst. (ICITEICS), Jun. 2024, pp. 1-6.

[16] A. Dey and S. Biswas, "Shot-ViT: Cricket batting shots classification with vision transformer network," Int. J. Eng., vol. 37, no. 12, pp. 2463-2472, 2024.

[17] D. Karmaker, A. Z. M. E. Chowdhury, M. S. U. Miah, M. A. Imran, and M. H. Rahman, "Cricket shot classification using motion vector," in Proc. 2nd Int. Conf. Comput. Technol. Inf. Manage. (ICCTIM), Apr. 2015, pp. 125-129.

[18] J. Donahue et al., "Long-term recurrent convolutional networks for visual recognition and description," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Jun. 2015, pp. 2625-2634.

[19] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," 2020, arXiv:2006.10204.

[20] Y. Yoon et al., "Analyzing basketball movements and pass relationships using realtime object tracking techniques based on deep learning," IEEE Access, vol. 7, pp. 56564-56576, 2019.

[21] G. Jin, "Player target tracking and detection in football game video using edge computing and deep learning," J. Supercomput., vol. 78, no. 7, pp. 9475-9491, May 2022.

[22] Y.-C. Li, C.-T. Chang, C.-C. Cheng, and Y.-L. Huang, "Baseball swing pose estimation using OpenPose," in Proc. IEEE Int. Conf. Robot., Autom. Artif. Intell. (RAAI), Apr. 2021, pp. 6-9.

[23] A. Ganser, B. Hollaus, and S. Stabinger, "Classification of tennis shots with a neural network approach," Sensors, vol. 21, no. 17, p. 5703, Aug. 2021.

[24] M. Goyani, S. K. Dutta, and P. Raj, "Key frame detection based semantic event detection and classification using hierarchical approach for cricket sport video indexing," in Proc. Int. Conf. Comput. Sci. Inf. Technol., 2010, pp. 388-397.

[25] K. R. Raval and M. M. Goyani, "A survey on event detection based video summarization for cricket," Multimedia Tools Appl., vol. 81, no. 20, pp. 29253-29281, Aug. 2022.

[26] H. Sattar, M. S. Umar, E. Ijaz, and M. U. Arshad, "Multi-modal architecture for cricket highlights generation: Using computer vision and large language model," in Proc. 17th Int. Conf. Open Source Syst. Technol. (ICOSST), Dec. 2023, pp. 1-6.

[27] I. Bhat, T. Sridhar, M. A. Yajur, and S. R. Upadhyaya, "Building a video dataset for cricket shot analysis," in Proc. Int. Conf. Netw., Multimedia Inf. Technol. (NMITCON), Sep. 2023, pp. 1-6.

[28] L. Wang et al., "Temporal segment networks for action recognition in videos," IEEE Trans. Pattern Anal. Mach. Intell., vol. 41, no. 11, pp. 2740-2755, Nov. 2019.

[29] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," 2024, arXiv:2410.17725.

[30] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" in Proc. Int. Conf. Mach. Learn. (ICML), 2021, pp. 813-824.

[31] Z. Liu et al., "Video Swin Transformer," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 3202-3211.

[32] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the Kinetics dataset," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 6299-6308.

[33] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 1, pp. 172-186, Jan. 2021.

[34] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in Proc. AAAI Conf. Artif. Intell., 2018, pp. 7444-7452.

[35] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 9459-9474.

[36] J. L. Fleiss, "Measuring nominal scale agreement among many raters," Psychol. Bull., vol. 76, no. 5, pp. 378-382, Nov. 1971.

[37] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2009, pp. 248-255.

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

[39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770-778.

[40] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

[41] PyTorch Documentation. [Online]. Available: https://docs.pytorch.org/docs/stable/

[42] OpenCV 4.x Documentation. [Online]. Available: https://docs.opencv.org/4.x/

[43] Google, "MediaPipe Pose Landmarker." [Online]. Available: https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

[44] FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[45] PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/

[46] Chart.js Documentation. [Online]. Available: https://www.chartjs.org/docs/latest/

[47] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2014, pp. 568-576.